CREACIÓN DE UN ROBOT ANTROPOMÓRFICO

ROBÓTICA

Elaborado por:

- María Villena Reviriego
- Iratxe Pérez Ribacoba
- Sebastián Martínez Maldonado
- Guillermo Rosales Pérez
- Manuel Sánchez Francés

ÍNDICE

1 INTRODUCCIÓN 5
1.1 ESTADO DEL ARTE DE LA ROBÓTICA 5
1.2 MORFOLOGÍA DEL ROBOT 6
1.2.1 Robot antropomórfico 7
2 DISEÑO DEL ROBOT 9
2.1 BOCETOS INICIALES Y DISEÑO EN AUTOCAD 9
2.2 PROCEDIMIENTOS Y ELEMENTOS UTILIZADOS 13
2.2.1 Impresión de las piezas del robot 13
2.2.2 Motores y controlador 13
2.2.3 Potenciómetros 14
2.2.4 Servomotor 15
2.2.5 Motor paso a paso 15
2.2.6 Placa controladora de accionamiento 16
2.3 MONTAJE DEL ROBOT 16
2.3.1 Gripper 17
2.3.2 Cuerpo del robot 18
3 CINEMÁTICA DEL ROBOT 21
3.1 CINEMÁTICA DIRECTA 21
3.2 CINEMÁTICA INVERSA 23
4 PROGRAMACIÓN DEL ARDUINO 26
4.1 PROGRAMACIÓN DE LA REALIMENTACIÓN 26
4.1.1 Problemas encontrados durante la programación de la realimentación de los servomotores 26
4.1.2 Problemas encontrados durante la programación de la realimentación del codo 28
4.1.3 Problemas encontrados durante la programación de la realimentación de la base 29
5 COSTES 31
6 REFERENCIAS 33
ANEXO I 34
ANEXO II 37

ÍNDICE DE FIGURAS

Figura 1: Máquina textil programable con tarjetas perforadas de Joseph Jacquard. 5
Figura 2: Primer robot comercial "Unimation". 6
Figura 3: (a) Robot PUMA y (b) Robot SCARA. 6
Figura 4: Sistema de cadena cerrada. 7
Figura 5: Robot antropomórfico, 8
Figura 6: Boceto inicial de robot. 9
Figura 7: Boceto de la realimentación con el potenciómetro. 9
Figura 8: Boceto final del robot. 10
Figura 9: (a) Anclaje izquierdo y (b) Anclaje derecho. 10
Figura 10: (a) Base rotatoria y (b) Base. 11
Figura 11: (a) Brazo 1 izquierdo y (b) Brazo 1 derecho. 11
Figura 12: (a) Brazo 2 izquierdo y (b) Brazo 2 derecho. 11
Figura 13: (a) Muñeca y (b) Eje del brazo 2. 12
Figura 14: Pinza. 12
Figura 15: Diseño 3D del robot completo. 12
Figura 16: Impresora 3D modelo Prusa i5. 13
Figura 17: Motor Nema17 y controlador A4988. 14
Figura 18: Servomotor del modelo SG90. 15
Figura 19: Motor paso a paso del modelo 28BYJ-48. 15
Figura 20: Placa controladora del modelo L298N. 16
Figura 21: Gripper ensamblado (sin motor). 17
Figura 22: Ensamblado del servomotor en la pinza. 17
Figura 23: Error de impresión en el anclaje derecho. 18
Figura 24: Incorrecta dimensión del eje del brazo 2. 18
Figura 25: Error de desacople de altura de la base. 18
Figura 26: Servomotor del modelo DM996. 19
Figura 27: Acople de Servomotor DM996 en lugar del motor paso a paso Nema 17. 19
Figura 28: Montaje completo del robot visto desde el lado izquierdo. 20
Figura 29: Montaje completo del robot visto desde el lado derecho. 20
Figura 30: Conversión de cinemática directa a inversa y viceversa. 21
Figura 31: Ejes, ángulos y longitudes del robot. 22
Figura 32: Indicación de un punto en la ventana de comandos de Matlab. 25
Figura 33: Representación gráfico del punto comandado en Matlab. 25
Figura 34: Diagrama de control de la realimentación. 26
Figura 35: Gráfica de fallo en la realimentación del servomotor. 27
Figura 36: Gráfica de error de convergencia en la realimentación del servomotor. 27
Figura 37: Gráfica con mejora en la realimentación del servomotor. 28
Figura 38: Gráfica de la realimentación del servomotor con un lazo de control adicional. 28
Figura 39: Gráfica de fallos en la realimentación del codo. 29
Figura 40: Gráfica de fallos en la realimentación del codo de la base giratoria. 29
Figura 41: Gráfica de fallos en la realimentación de la base. 30
Figura 42: Gráfica del delay de la base. 30

ÍNDICE DE TABLAS

Tabla 1: Características de la impresora 3D. 13
Tabla 2: Características del motor Nema 17. 14
Tabla 3: Características del controlador A4988. 14
Tabla 4: Características del servomotor SG90. 15
Tabla 5: Características del motor paso a paso 28BYJ-48. 16
Tabla 6: Características de la placa controladora L298N. 17
Tabla 7: Características del servomotor DM996. 19
Tabla 8: Parámetros del método Denavit-Hartenberg. 22
Tabla 9: Parámetros Denavit-Hartenberg del robot. 23
Tabla 10: Costes de la impresión de las piezas del robot. 31
Tabla 11: Costes de los elementos electrónicos. 31
Tabla 12: Costes de materiales adicionales para el montaje del robot. 32

1 INTRODUCCIÓN

El trabajo que se presenta en este informe es la realización de un robot antropomórfico, desde el diseño del mismo hasta los cálculos cinemáticos y la programación que permiten su funcionamiento. El objetivo es que el robot sea capaz de construir una pared vertical de bloques de 80x40x40 mm.

Para conseguir esto, en primer lugar, se ha utilizado el programa AutoCAD para diseñar cada una de las partes que componen el robot y se han impreso con una impresora 3d. Una vez conocidas las dimensiones y peso del robot, se han elegido los elementos que permiten su movimiento, como los motores y el potenciómetro, entre otros. El siguiente paso ha sido calcular la cinemática directa e inversa que permiten, por un lado, conocer la postura (posición y orientación) del efector final del robot, con respecto a su base y, por otro lado, determinar las configuraciones de articulaciones necesarias para que el modelo de robot logre la posición del efector final deseada. Estos cálculos se han desarrollado con el programa Matlab. El último paso ha consistido en programar el Arduino, lo que ha permitido controlar y alimentar los diferentes motores del robot, consiguiendo que este se mueva de la forma deseada.

A lo largo del desarrollo del proyecto han surgido dificultades, desde problemas en la impresión de las piezas que se han solucionado rediseñando las mismas para que se adapten a la funcionalidad del robot adecuadamente, problemas en la elección de los motores, que no tenían la fuerza suficiente para mover las articulaciones del robot, hasta problemas en la realimentación (hardware y potenciómetro).

1.1 ESTADO DEL ARTE DE LA ROBÓTICA

El comienzo de la era moderna de la robótica puede trazarse hasta la revolución textil del siglo XVIII. En 1801, Joseph Jacquard introdujo una máquina textil que podía programarse utilizando tarjetas perforadas (Figura 1). Este hito marcó el inicio de una serie de avances impulsados por la Revolución Industrial. Entre ellos se destacan el torno mecánico de Spencer en 1830, el brazo motorizado de Babbitt en 1892 y el mecanismo programable para pintar con spray creado por Pollard y Roselund en 1938.

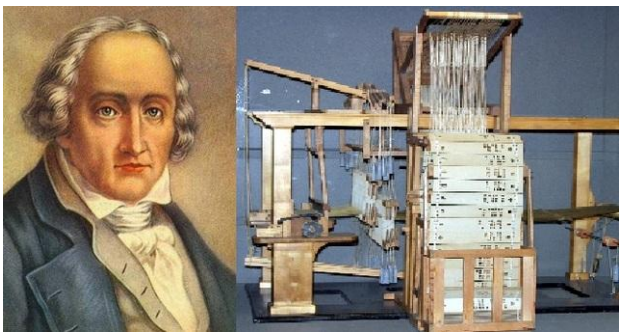


Figura 1: Máquina textil programable con tarjetas perforadas de Joseph Jacquard.

Las primeras patentes específicas para robots surgieron en 1946 con los rudimentarios robots de Devol diseñados para la transferencia de maquinaria. Ese mismo año también vio el surgimiento de las primeras computadoras: J. Presper Eckert y John Mauchly construyeron el ENIAC en la Universidad de Pensilvania, mientras que en el MIT se desarrolló la primera máquina digital de propósito general.

En 1954, Devol introduce el primer robot programable y acuña el término "autómata universal", que más tarde recorta a "Unimation" (Figura 2). Esta sería la denominación que Engleberger elegiría para la primera empresa de robótica. La comercialización de robots daría inicio en 1959 con el primer modelo de la Planet Corporation. En 1962, General Motors incorpora el primer robot industrial en su línea de producción en Trenton, Nueva Jersey. En 1964, se establecen laboratorios de investigación en

inteligencia artificial en el MIT, el SRI (Instituto de Investigación de Stanford) y en la Universidad de Edimburgo.

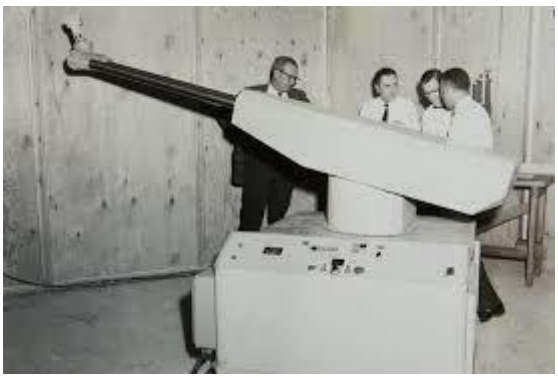
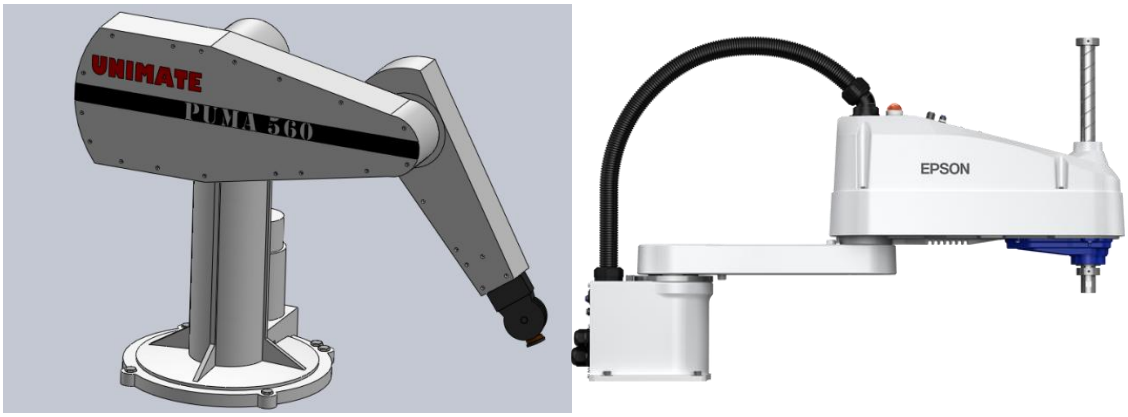


Figura 2: Primer robot comercial "Unimation".

Poco después, los japoneses, que previamente habían dependido de la tecnología robótica importada, emergen como líderes en el mercado. La Kawasaki Heavy Industries firma un acuerdo de colaboración con Unimation. En 1970, se desarrolla un brazo mecánico para investigaciones en Stanford. Su inventor, Victor Scheinman, establece Vicarm Inc en 1974 para comercializar el prototipo, el cual estaba controlado por un minicomputador. En 1976, tanto la sonda Viking 1 como la 2 implementan brazos robóticos, y Vicarm integra microcomputadores en sus diseños. Al mismo tiempo, la NASA inicia un programa de colaboración con el Jet Propulsion Laboratory (JPL) para el desarrollo del MARSROVER. En 1977, Unimation adquiere Vicarm y un año después lanzan el PUMA (Programmable Universal Manipulator for Assembly). Posteriormente, IBM y Seiki se unen para crear el SCARA, que poseía el 85 % de las capacidades del PUMA pero a mitad de precio (Figura 3). [1]



(a) (b)

Figura 3: (a) Robot PUMA y (b) Robot SCARA.

1.2 MORFOLOGÍA DEL ROBOT

La clasificación de un robot se basa en el tipo de articulaciones que posea, pudiendo ser rotacionales, que permiten sólo movimientos de rotación, o prismáticas o lineales, que generan desplazamientos longitudinales. La combinación de estas articulaciones determina el tipo de robot industrial. En este estudio, el enfoque está en el robot antropomórfico, que cuenta con al menos 3 articulaciones rotacionales.

Para ubicar el robot en el espacio tridimensional, se necesitan 6 coordenadas: tres para su posición en el sistema cartesiano y tres para la orientación de la herramienta de trabajo. La relación entre estas coordenadas, las articulaciones y su orientación se denomina cinemática directa.

Cuando se aborda la morfología del robot, se describe su estructura mecánica, componentes y partes. Básicamente, un robot es un sistema versátil que puede realizar diversas tareas, como transporte de objetos, pintura, soldadura, ensamblaje, entre otras. En su esencia, un robot consta de una serie de eslabones y articulaciones que forman una cadena cinemática abierta, donde los extremos no están unidos formando un bucle cerrado.

El robot está constituido por una articulación inicial que sirve como base, seguida por una serie de eslabones y articulaciones. En el extremo, se encuentra un espacio destinado para colocar una herramienta que permite realizar tareas específicas. Cada articulación proporciona un grado de libertad y transmite energía para generar movimiento en los eslabones conectados. Los eslabones están compuestos por barras rígidas que están mecánicamente unidas al rotor de una articulación y al estator de la siguiente.

El espacio de trabajo de un robot se define como el área donde puede realizar sus movimientos posibles, determinado por su geometría y tipo de articulaciones. La representación del posicionamiento del robot implica un sistema de coordenadas cartesianas que define su posición y orientación. La transformación homogénea es una herramienta que relaciona las operaciones de rotación y traslación en una matriz, facilitando el análisis de la cinemática directa. [2]

En cuanto al control del robot, esto se hace a través de un sistema conocido como cadena cerrada. Este es aquel en el que la salida tiene un efecto sobre la señal de control y, por tanto, la señal de error actuadora (que es la diferencia entra la señal de entrada y la de realimentación) que entra al controlador para reducir el error y llevar a la salida al valor deseado. La señal de realimentación puede ser igual a la salida o una función de sus derivadas.

Un esquema de este tipo de control se puede observar en la siguiente figura (Figura 4), en la que se aprecia una función de transferencia G(s) sobre la que actúa la señal de error e(t), y una función de realimentación H(s) que es función de la salida c(t). La señal de error es la diferencia entre la señal de referencia y la señal de realimentación.

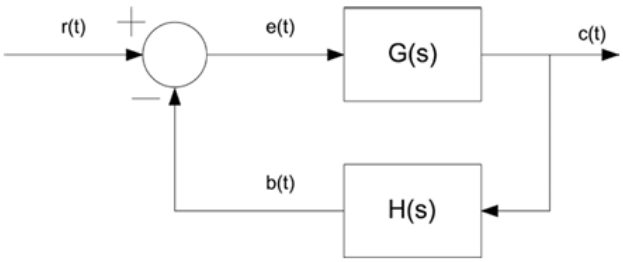


Figura 4: Sistema de cadena cerrada.

1.2.1 Robot antropomórfico

La palabra “antropomórfico” se refiere a cualquier cosa que tenga el aspecto físico parecido al del ser humano, por lo tanto, “robot antropomórfico” se es un robots que se asemeja al ser humano principalmente en la parte del hombro, brazo y muñeca, lo que permite posicionarse en el espacio y orientarse para realizar un determinado trabajo. Este tipo de configuración de robots industriales son empleados en tareas pesadas y repetitivas, aprovechando la variedad de movimientos que posee.

Cuentan con 3 articulaciones de posicionamiento y, por general, 3 articulaciones de orientación, es decir, para el efector final, que puede ser una pinza o gripper.

Figura 5: Robot antropomórfico,

En cuanto al número y tipo de articulaciones, este robot está formado por 3 ejes rotacionales llamados también 3G o RRR, el primero, perpendicular al suelo y, los otros dos, perpendiculares al primero y paralelos entre sí. Su funcionalidad es posicionar al robot en un determinado lugar del espacio.

Otra de las características de los robots antropomórficos son los grados de libertad o GDL. Estos se refieren al número mínimo de variables necesarias para describir la distribución de velocidades en el sistema. En este caso, se tienen generalmente 5 o 6 grados de libertad, tres por lo menos usados para definir la posición y mínimo dos para la orientación de la pinza. [3]

2 DISEÑO DEL ROBOT

En este apartado se muestran, por un lado, los primeros pasos del diseño del robot, desde las ideas iniciales plasmadas en los bocetos hasta el diseño completo en AutoCAD y, por otro lado, los elementos utilizados en la construcción del robot, como máquinas, motores, potenciómetros y servomotores.

2.1 BOCETOS INICIALES Y DISEÑO EN AUTOCAD

El primer paso en el diseño del robot ha sido diseñar una serie de bocetos que permitan establecer las características fundamentales, tratadas en el apartado anterior, y que sirvan de guía en los pasos siguientes de su construcción. Estos parámetros son, el número de grados de libertad, en este caso, 4, con un controlador abre-cierra (Gripper) en la punta. En la siguiente imagen (Figura 6) se muestra el boceto inicial.

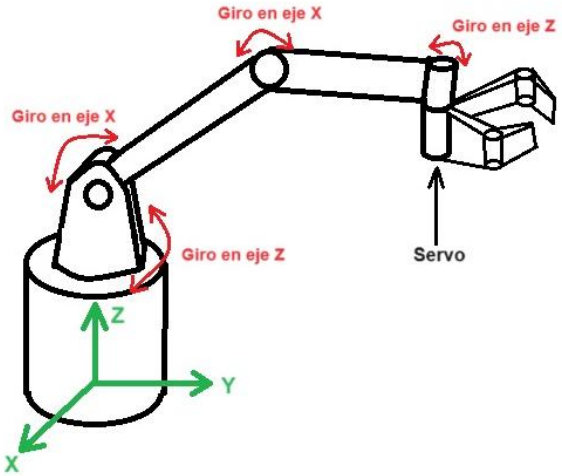


Figura 6: Boceto inicial de robot.

Tras varios análisis de la idea inicial, surgen otros bocetos que mejoran el diseño del robot. Estos nuevos diseños incorporan un motor en la articulación más próxima a la pinza, con el objetivo de proporcionarle más fuerza. Además, también se añade un potenciómetro en el eje de la articulación 1, como se muestra en la siguiente imagen (Figura 7).

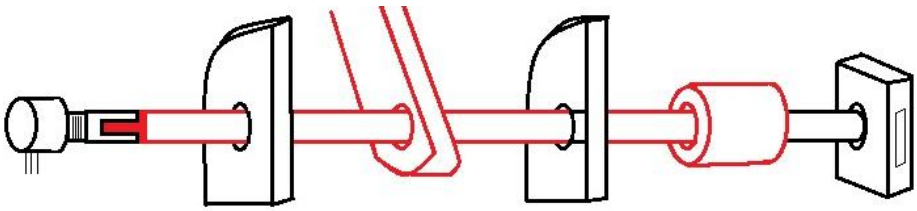


Figura 7: Boceto de la realimentación con el potenciómetro.

Este análisis previo da lugar al boceto final (Figura 8), que sirve de guía en el diseño de las piezas en el programa AutoCAD. Como se puede apreciar, finalmente, se instalan dos potenciómetros, un motor paso a paso en la pinza y otro en la articulación 2.

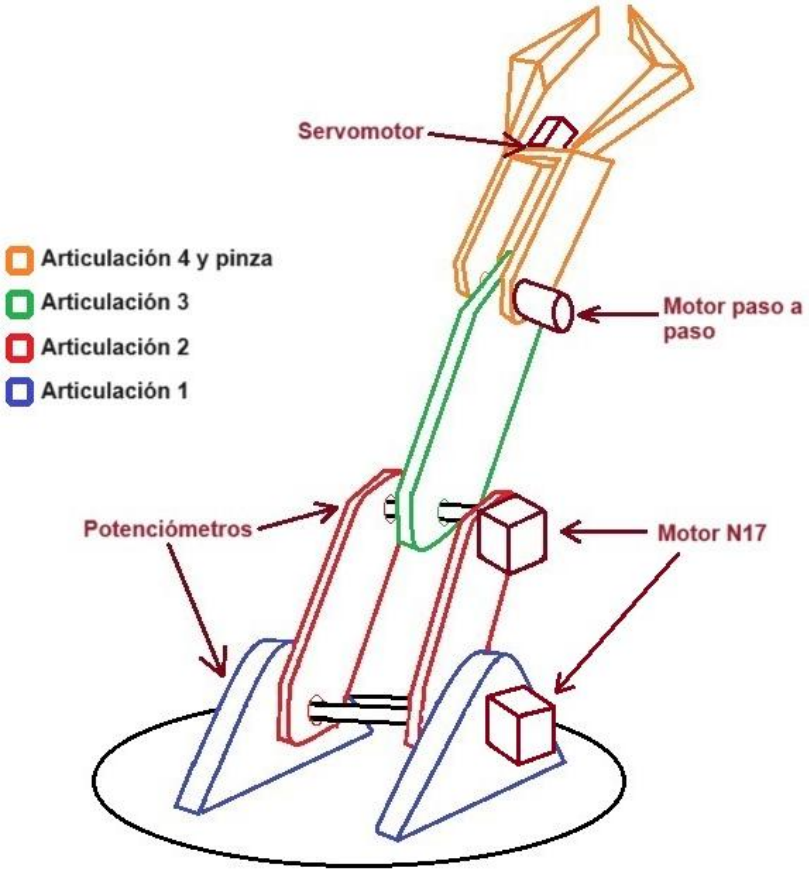
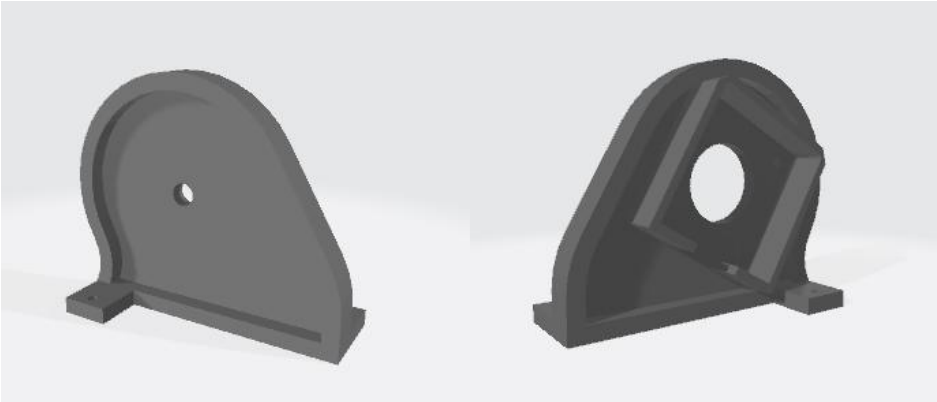


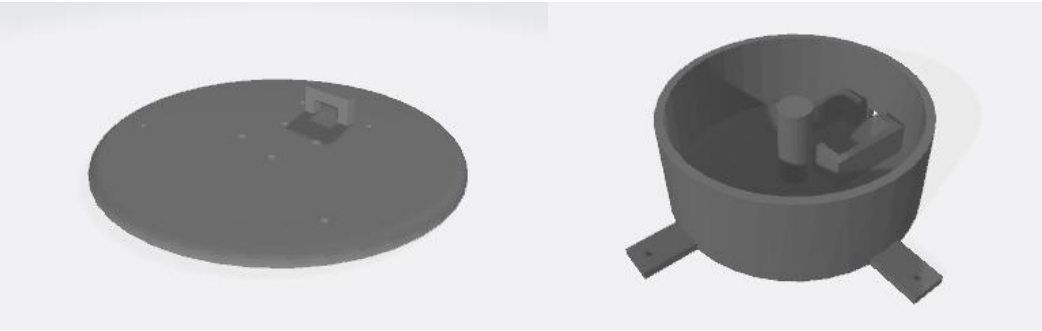
Figura 8: Boceto final del robot.

Cada una de las piezas mostradas en la imagen del boceto final se han diseñado en el programa AutoCAD para, después, ser impresas en una impresora 3d. A continuación se muestra cada una de las piezas modeladas:



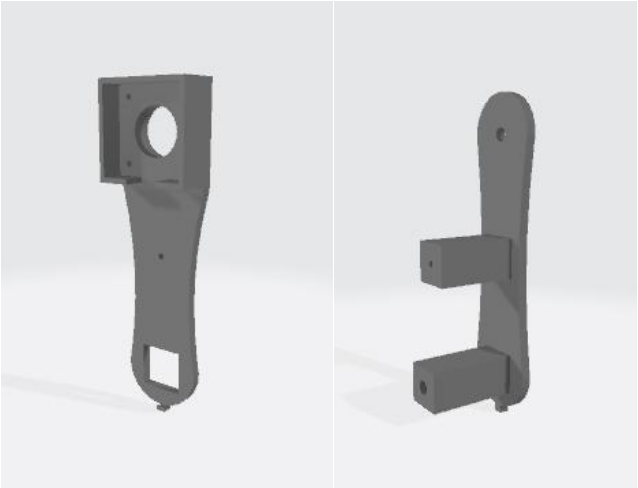
(a) (b)

Figura 9: (a) Anclaje izquierdo y (b) Anclaje derecho.



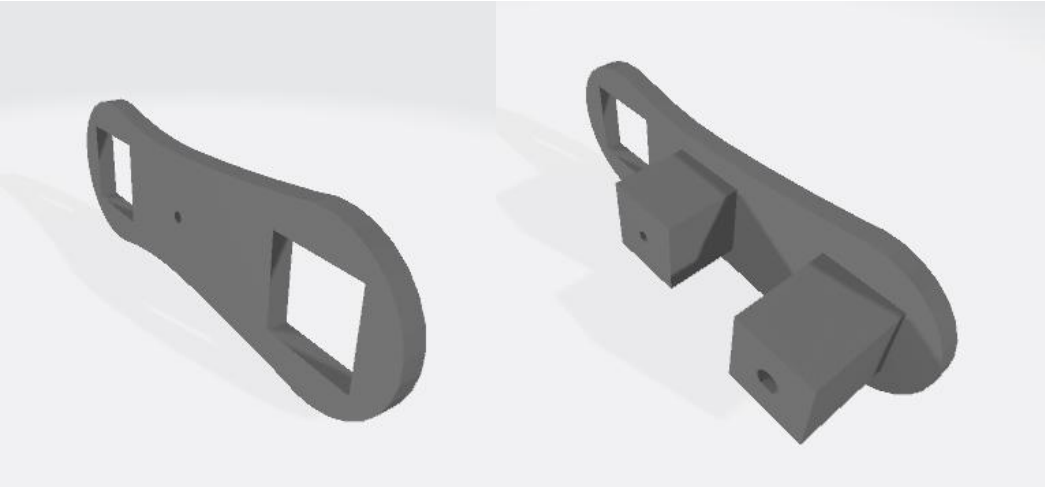
(a) (b)

Figura 10: (a) Base rotatoria y (b) Base.



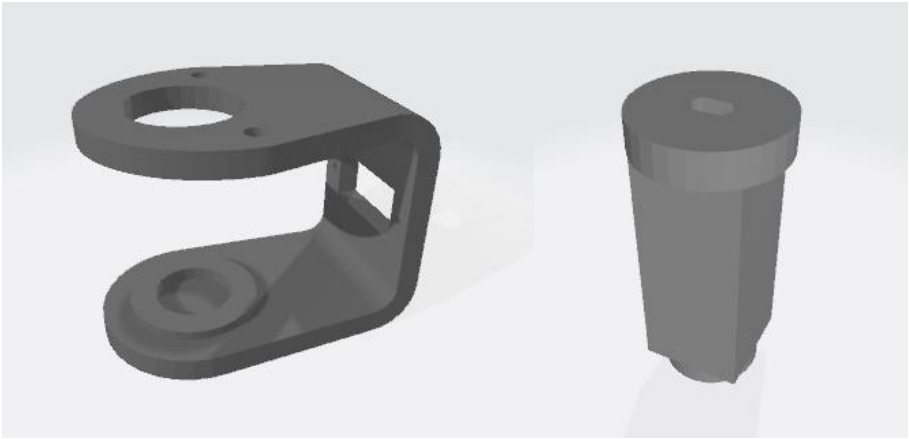
(a) (b)

Figura 11: (a) Brazo 1 izquierdo y (b) Brazo 1 derecho.



(a) (b)

Figura 12: (a) Brazo 2 izquierdo y (b) Brazo 2 derecho.



(a) (b)

Figura 13: (a) Muñeca y (b) Eje del brazo 2.

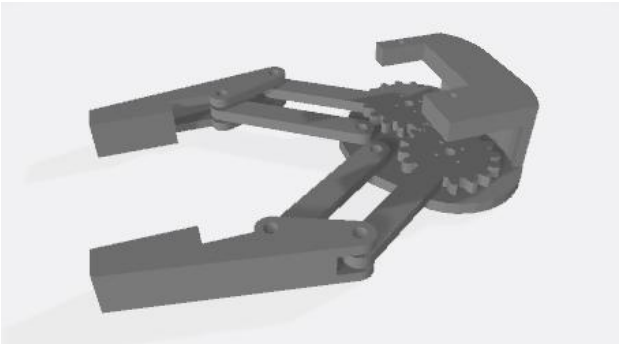


Figura 14: Pinza.

Dando como resultado la composición completa del robot, mostrado en la siguiente imagen:

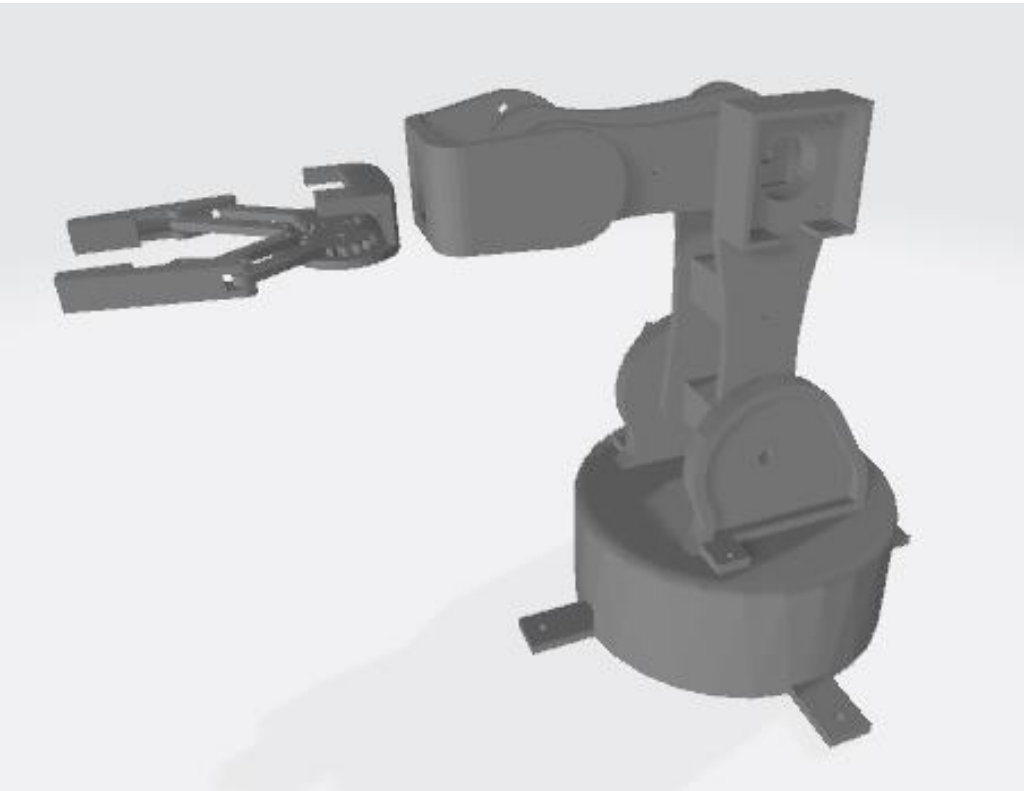


Figura 15: Diseño 3D del robot completo.

2.2 PROCEDIMIENTOS Y ELEMENTOS UTILIZADOS

En este apartado se describen los elementos utilizados para la construcción física del robot, desde la impresión de las partes del robot hasta los motores, servomotores y potenciómetros, entre otros.

2.2.1 Impresión de las piezas del robot

Para la impresión de las diferentes piezas que componen el robot se utiliza una impresora 3D. Concretamente el modelo Prusa i5 con el material PLA en color negro. Esta impresora utiliza el método FDM (Fused Deposition Modeling), que consiste en lo siguiente: el filamento de plástico (en este caso PLA) se alimenta desde un carrete a través del extrusor, donde se derrite a alta temperatura. El extrusor se mueve a lo largo de los ejes X, Y y Z según las instrucciones del archivo de impresión, depositando capas del material derretido en la plataforma de impresión. A medida que se depositan las capas de material fundido, se enfrían y solidifican rápidamente, formando una estructura sólida.

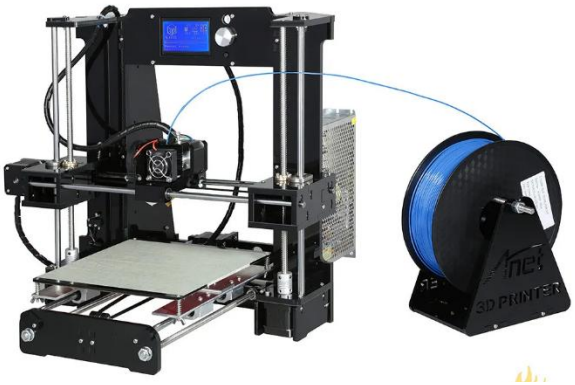


Figura 16: Impresora 3D modelo Prusa i5.

Tabla 1: Características de la impresora 3D.

Característica	Medida
Potencia	110 V ~ 240 V 50/60 Hz
Grosor de las capas	0,1-0,4 mm
Precisión de posicionamiento del eje Z	0,004 mm
Velocidad de impresión	100 mm/s
Resolución máxima	0,1 mm
Precisión de posicionamiento de ejes XY	0,012 mm
Tamaño de impresión	220 mm x 220 mm x 240 mm
Diámetro de la boquilla	0,3/0,35/0,4 mm

2.2.2 Motores y controlador

Los motores utilizados en un principio para controlar cada grado de libertad del robot son los Nema17 paso a paso del modelo 17HS4023. De igual manera, se elige el uso de un módulo de control A4988 para los motores (Figura 4).

Los motores permiten que cada articulación del brazo se mueva en diferentes direcciones según el diseño y la configuración del brazo, además, proporcionan control y la velocidad a la que se mueve el brazo y su capacidad para responder rápidamente a cambios en el entorno o instrucciones de operación.

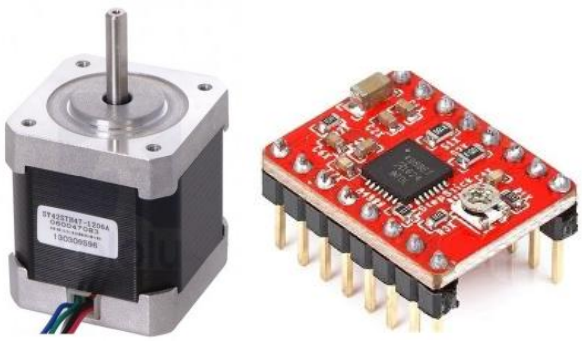


Figura 17: Motor Nema17 y controlador A4988.

En la siguiente tabla se presentan las características del motor:

Tabla 2: Características del motor Nema 17.

Característica	Medida
Tipo motor	Bipolar
Diámetro del eje	5 mm
Longitud del eje	25 mm
Tamaño	42 x 42 x 23 mm
Ángulo del paso	1,8 º (200 pasos por vuelta)
Intensidad máxima por fase	0,7 A
Voltaje por fase	4 V

En la siguiente tabla se presentan las características del controlador:

Tabla 3: Características del controlador A4988.

Característica	Medida
Tamaño	20 x 15 x 11 mm
Voltaje	5 V
Peso	18,10 g

2.2.3 Potenciómetros

Junto con los motores, para añadir la realimentación, se elige hacer uso de potenciómetros. Un potenciómetro es un dispositivo electrónico que consiste en una resistencia variable que se ajusta manualmente mediante un control giratorio o deslizante, es decir, un punto de contacto móvil que puede deslizarse a lo largo de la resistencia fija. Al cambiar la posición del punto de contacto, se modifica la resistencia eléctrica entre dos terminales.

En el caso del robot, el potenciómetro se utiliza para medir la posición de una articulación, de manera que se pueda controlar el movimiento y la posición del robot.

Se han usado dos potenciómetros para controlar las articulaciones 1 y 2, como se muestran en la Figura 8.

2.2.4 Servomotor

Para proporcionar movimiento a la pinza del robot, se ha implementado un servomotor del modelo SG90, ya que permite un control preciso sobre su posición y su movimiento, lo que es crucial para la aplicación de este robot, que es construir una pared de bloques. Además también se puede controlar la fuerza ejercida.

Figura 18: Servomotor del modelo SG90.

Las características del servomotor del modelo SG90 son las siguientes:

Tabla 4: Características del servomotor SG90.

Característica	Medida
Torque	1.3 Kg.cm (4.8 V), 1.6 Kg.cm (6.0 V)
Voltaje de operación	4.8 V (~5 V)
Velocidad de Operación (4.8V sin carga)	0.1 s/60 grados
Tamaño	22.2 x 11.8 x 31 mm
Peso	9 gramos
Angulo de Rotación	180° máximo
Material engranajes	Nylon
Pulso ciclo	2.4 ms
Ancho del pulso	4 uS (Dead band)
Rango de Temperatura	0 ºC – 55 ºC

2.2.5 Motor paso a paso

Un motor paso a paso es un tipo de motor eléctrico que convierte pulsos eléctricos en movimientos discretos angulares, conocidos como pasos. Cada pulso eléctrico provoca un paso angular preciso del eje del motor. Para el brazo robot de este proyecto se ha optado por escoger el motor paso a paso del modelo 28BYJ-48.

Figura 19: Motor paso a paso del modelo 28BYJ-48.

En un brazo robótico, los motores paso a paso pueden usarse para controlar los movimientos de las articulaciones, permitiendo al robot alcanzar y manipular objetos con precisión. Por ejemplo, se pueden utilizar en las articulaciones de hombro, codo y muñeca, como es este caso, para controlar la orientación y la posición del extremo del brazo robótico. Además, su capacidad para proporcionar un alto par de torsión los hace adecuados para levantar y manipular cargas.

Las características del motor paso a paso del modelo 28BYJ-48 son las siguientes:

Tabla 5: Características del motor paso a paso 28BYJ-48.

Característica	Medida
Ángulo de paso eje	0.087º/paso – 4096 pasos / vuelta
Voltaje de operación	12 V
Resistencia	200 Ohm +/-7% (por bobina)
Tamaño	D28 x 19 mm
Par de bloqueo	34.3 mN·m / 34.3 g · cm / 0.343 N·cm
Ángulo por paso	5,625º/64.
Torque	0,3 Kgr/cm
Rango de Temperatura	-45 ºC a +125 ºC.
Frecuencia	100 Hz
Fases motor	4

2.2.6 Placa controladora de accionamiento

Una placa controladora de accionamiento, también conocida como controlador de motor, es un componente electrónico utilizado para controlar motores eléctricos en aplicaciones robóticas y de automatización.

2.3 MONTAJE DEL ROBOT

Una vez impresas las piezas y elegidos los elementos electrónicos, se procede a montar cada parte correspondiente del robot. A lo largo de este proceso y tras realizar una serie de pruebas de funcionamiento, se detectan varios errores que se comentan a continuación.

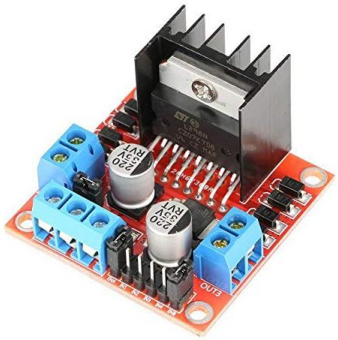


Figura 20: Placa controladora del modelo L298N.

En este caso se utiliza para controlar los motores que accionan las articulaciones del robot.

Tabla 6: Características de la placa controladora L298N.

Característica	Medida
Chip controlador	controlador de puente H dual L298N
Voltaje de accionamiento	+5 V a +35 V
Corriente máxima de la unidad	2A/puente
Rango de fuente de alimentación lógica Vss	+4 V a +5,5 V CC
Rango de voltaje de entrada de señal de control	4.5-5.5 V alto 0 V bajo
Consumo máximo de energía	20 W
Temperatura de almacenamiento	-25 ºC ~ +130 ºC
Dimensiones	43 mm x 43 mm x 27 mm
Peso	26 g

2.3.1 Gripper

Para el ensamblado del gripper o pinza, se unen las diferentes piezas impresas con tornillos M3. Durante el ajuste, se detecta que faltan dos agujeros para completar el ensamble completo. Para solucionar esto, y evitar malgastar tiempo y dinero al volver a imprimir la pieza, se realizan dos taladrados en esos dos puntos.

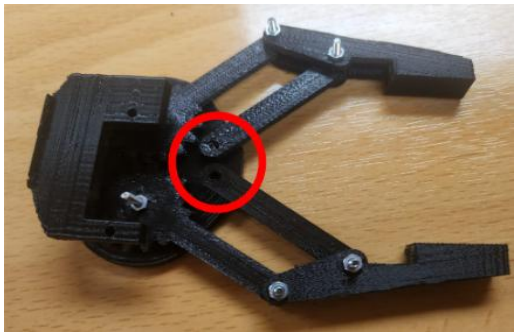


Figura 21: Gripper ensamblado (sin motor).

Una vez solucionado el problema, se procede a acoplar el servomotor escogido para la pinza, como se muestra en la siguiente imagen.

Figura 22: Ensamblado del servomotor en la pinza.

2.3.2 Cuerpo del robot

En el caso del cuerpo del robot, ocurren distintos problemas con las piezas, ya que no solo existen problemas de impresión, sino que se presentan problemas de dimensión dentro de las piezas, como se observa en las siguientes figuras:

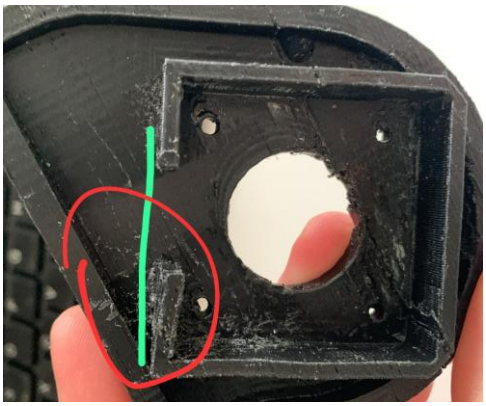


Figura 23: Error de impresión en el anclaje derecho.

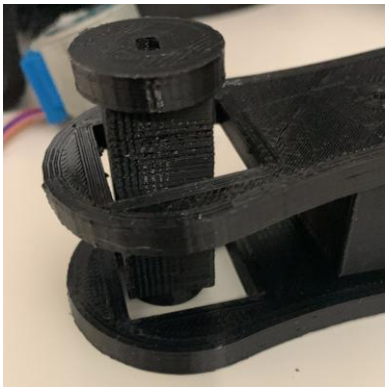


Figura 24: Incorrecta dimensión del eje del brazo 2.

Además de estos problemas, también hay errores al acoplar los motores a la base, ya que hay un desacople de altura con la base fija y la base rotatoria del robot:

Figura 25: Error de desacople de altura de la base.

Al hacer una prueba de movimiento entre dos brazos del robot, se detecta que el motor elegido, Nema 17, no da la suficiente fuerza para mantener su posición correctamente. Como solución se opta por cambiar los motores por otros que tengan la suficiente fuerza para mover las articulaciones. Después de revisar las diferentes opciones, se escogen los servomotores DIY MORE, de modelo DM996, que

cuenta con un par mayor que los anteriores motores, con la ventaja de que su programación resulta un más sencilla que la de los motores paso a paso.

Figura 26: Servomotor del modelo DM996.

Las características del servomotor del modelo MG996R son las siguientes:

Tabla 7: Características del servomotor DM996.

Característica	Medida
Torque	9.4 kg/cm (4.8 V), 11 kg/cm (6 V)
Voltaje de operación	4.8 – 7.2 V
Velocidad de Operación (4.8V sin carga)	0.2 seg / 60 grados
Velocidad de Operación (6V sin carga)	0.16 seg / 60 grados
Tamaño	40,6 x 19,8 x 42,9 mm
Peso	55 g
Angulo de Rotación	180° máximo
Material engranajes	Metal
Pulso ciclo	20 mS
Ancho del pulso	entre 600 uS y 2400 uS
Rango de Temperatura	-30 a +60 °C

Como se puede apreciar comparando las dimensiones del motor elegido en un inicio, Nema 17, y el motor utilizado finalmente, MG996R, las del primero son mayores que las del segundo, por lo tanto, para evitar retrasos y costes adicionales, se opta por modificar las piezas del robot en donde se acopla cada motor.

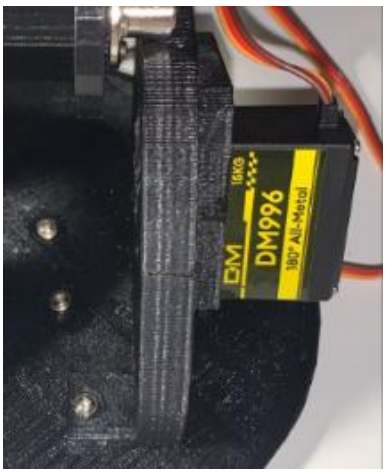


Figura 27: Acople de Servomotor DM996 en lugar del motor paso a paso Nema 17.

Finalmente se consigue el montaje final del robot, que se puede apreciar en las siguientes imágenes:

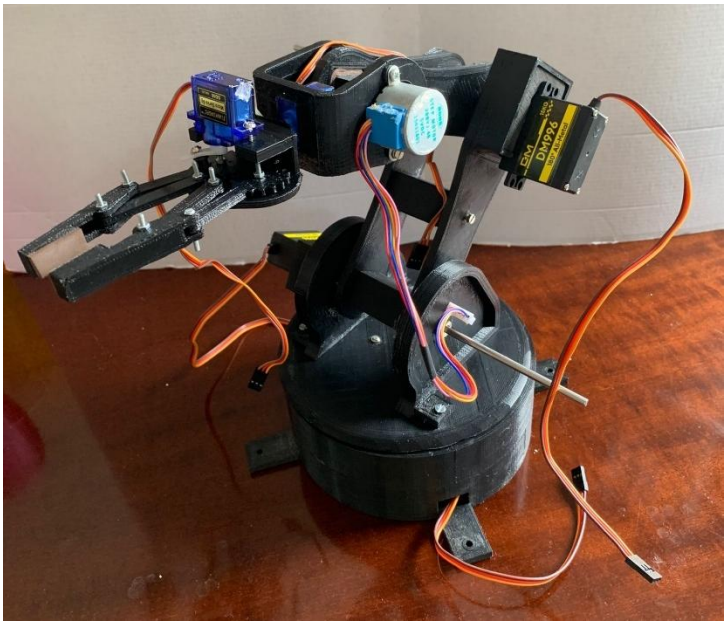


Figura 28: Montaje completo del robot visto desde el lado izquierdo.

Figura 29: Montaje completo del robot visto desde el lado derecho.

3 CINEMÁTICA DEL ROBOT

La cinemática del robot estudia el movimiento del mismo con respecto a un sistema de referencia fijo sin considerar las fuerzas y momentos que originan dicho movimiento. Busca las relaciones entre la localización (posición y orientación) del extremo del robot y los valores de sus coordenadas articulares. Busca las relaciones entre las velocidades del movimiento de las articulaciones y el extremo.

- Cinemática directa
  Determina la localización del extremo del robot, con respecto a un sistema de coordenadas de referencia, conocidos los valores de las articulaciones y los parámetros geométricos de los elementos del robot.

- Cinemática inversa
  Conocida la localización del robot, determina cuál debe ser la configuración del robot (articulaciones y parámetros geométricos). [3, 4]

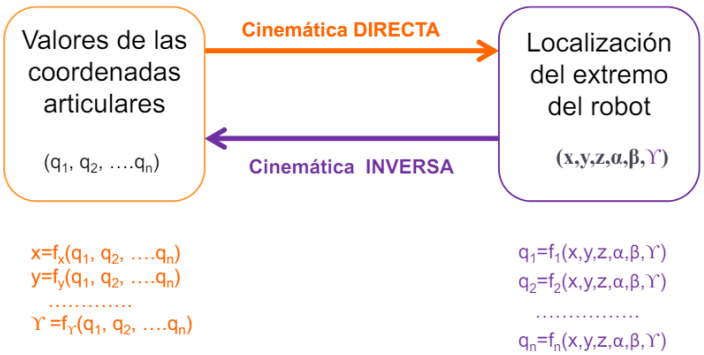


Figura 30: Conversión de cinemática directa a inversa y viceversa.

3.1 CINEMÁTICA DIRECTA

La cinemática directa consiste en determinar la posición y orientación del efector final en función de las variables de las articulaciones. En este caso, para resolverla, se emplea el método de Devanit-Hartenberg.

El método de Denavit-Hartenberg (DH) es un enfoque sistemático utilizado para describir la cinemática de un robot manipulador. Este método proporciona una forma estándar de establecer un sistema de coordenadas local para cada articulación del robot, lo que facilita el análisis y la representación de su configuración espacial y movimiento.

Este método consiste en los siguiente pasos:

- Definición de los ejes de las articulaciones: se asigna un sistema de coordenadas local a cada articulación del robot. Esto implica definir ejes de rotación o traslación para cada junta, lo que determina cómo se mueve una articulación con respecto a la siguiente.
- Asignación de los ejes de las articulaciones: se asignan valores de parámetros DH (a, d, α, θ) a cada par de juntas adyacentes. Estos parámetros representan las distancias y ángulos entre los ejes de las articulaciones, así como las rotaciones entre los sistemas de coordenadas locales.
- Construcción de las matrices de transformación: utilizando los parámetros DH asignados, se construyen matrices de transformación homogéneas para cada articulación. Estas matrices representan las transformaciones geométricas entre los sistemas de coordenadas locales de las articulaciones adyacentes.

- Cálculo de la matriz de transformación total: se multiplican las matrices de transformación homogéneas de cada articulación para obtener una matriz de transformación total que describe la configuración espacial completa del robot.
- Análisis cinemático: con la matriz de transformación total, se pueden analizar varios aspectos cinemáticos del robot, como su posición, orientación, trayectorias y espacio de trabajo. [4,5]

Siguiendo los pasos del método de Denavit-Hartenberg, se definen los ejes, ángulos y longitudes de cada articulación como se muestra en la siguiente imagen:

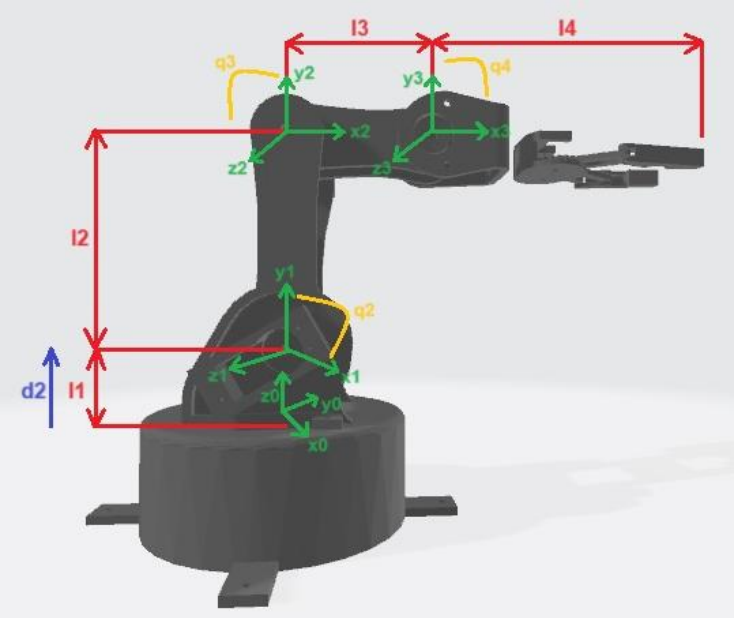


Figura 31: Ejes, ángulos y longitudes del robot.

El siguiente paso es la creación de una tabla siguiendo las siguientes pasos:

- Obtener θ_i como el ángulo que hay que girar en torno a Z_{i-1} para que X_{i-1} y X_i queden paralelos.
- Obtener di como la distancia medida a lo largo de Z_{i-1}, que habría que desplazar $\{S_{i-1}\}$ para que X_i y X_{i-a} queden en el mismo plano.
- Obtener a_i como la distancia medida a lo largo de X_i (que ahora coincidiría con X_{i-1}) que habría que desplazar el nuevo $\{S_{i-1}\}$ para que su origen coincidiese con $\{S_i\}$.
- Obtener αi como el ángulo que habría que girar en torno a X_i (que ahora coincidiría con X_{i-1}) para que el nuevo $\{S_{i-1}\}$ coincidiese totalmente con $\{S_i\}$. [4]

En la tabla 6 se observan los parámetros Denavit-Hartenberg obtenidos para el robot:

Tabla 8: Parámetros del método Denavit-Hartenberg.

Articulación	$\boldsymbol{\theta_i}$	$\boldsymbol{d_i}$	$\boldsymbol{a_i}$	$\boldsymbol{\alpha_i}$
1	q_1	d_1	l_1	90º
2	q_2	0	l_2	0º
3	q_3	0	l_3	0º
4	q_4	0	l_4	0º

Una vez se han hallado todos los parámetros de Devanit-Hartenberg, se obtienen las matrices de transformación, que siguen la siguiente expresión:

$$A_{\mathrm{i}}^{\mathrm{i}-1} = \begin{bmatrix} cos\theta_{\mathrm{i}} & -\cos\alpha_i * sen\theta_i & sen\alpha_i * sen\theta_{\mathrm{i}} & a_1 * cos\theta_{\mathrm{i}} \\ sen\theta_{\mathrm{i}} & \cos\alpha_i * cos\theta_{\mathrm{i}} & -sen\alpha_i * cos\theta_{\mathrm{i}} & a_1 * sen\theta_{\mathrm{i}} \\ 0 & \mathrm{sen}\alpha_i & 0 & d_{\mathrm{i}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_1^0 = \begin{bmatrix} cosq_1 & 0 & senq_1 & l_1 * cosq_1 \\ senq_1 & 0 & -cosq_1 & l_1 * senq_1 \\ 0 & 1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2^1 = \begin{bmatrix} cosq_2 & -senq_2 & 0 & l_2 * cosq_2 \\ senq_2 & cosq_2 & 0 & l_2 * senq_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3^2 = \begin{bmatrix} cosq_3 & -senq_3 & 0 & l_3 * cosq_3 \\ senq_3 & cosq_3 & 0 & l_3 * senq_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_4^3 = \begin{bmatrix} cosq_4 & -senq_4 & 0 & l_4 * cosq_4 \\ senq_4 & cosq_4 & 0 & l_4 * senq_4 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Por último, la matriz de transformación (T) entre la base y el extremo del robot se obtiene multiplicando todas las matrices de transformación entre 0 y 4.

$$T = A_1^0 * A_2^1 * A_3^2 * A_4^3 = A_4^0$$

3.2 CINEMÁTICA INVERSA

Conocida la localización del robot, la cinemática inversa determina cuál debe ser la configuración del robot, es decir, los valores que deben adoptar las coordenadas articulares del robot (q_1, q_2, q_3 y q_4) para que su extremo final se posicione y oriente según una determinada localización espacial.

Su resolución no es sistemática ni cerrada, ya que depende de la configuración del robot. El objetivo es encontrar la siguiente relación explícita:

$$q_k = f_k(x, y, z, \alpha, \beta, \gamma) \qquad k = 1 \ldots n\ (GDL)$$

En este caso, como el robot tiene cuatro grados de libertad, k = 4.

Para la resolución de la cinemática inversa se emplearán métodos geométricos. El objetivo principal es encontrar un número suficiente de relaciones geométricas y trigonométricas que involucren tanto a las coordenadas del extremo del robot (x, y, z y α) como a sus coordenadas articulares (q_1, q_2, q_3 y q_4) y a las dimensiones físicas de los eslabones (L_1, L_2, L_3 y L_4).

Tabla 9: Parámetros Denavit-Hartenberg del robot.

Articulación	$\boldsymbol{\theta_i}$	$\boldsymbol{d_i}$	$\boldsymbol{a_i}$	$\boldsymbol{\alpha_i}$
1	q_1	d_1	0,105	90º
2	q_2	0	0,120	0º
3	q_3	0	0,082	0º

4	q_4	0	0,145	0º

A continuación se expresan las ecuaciones con las que se calculan los valores de q_1, q_2, q_3 y q_4 que, en las diferentes iteraciones que se realicen, dan un valor u otro.

- Ángulo theta1 (rotación de la base):

$$\theta_1 = tan^{-1}\left(\frac{x}{y}\right)$$

- Distancia horizontal desde la base al punto proyectado en XY:

$$D = \sqrt{x^2 + y^2} - L_4 * cos(0)$$

- Altura vertical desde la base al punto P:

$$Z = z - L_1 - L_4 * sen(0)$$

- Distancia total desde la base al punto final P:

$$J = \sqrt{D^2 + Z^2}$$

- Ángulo theta3 (rotación del codo) usando la ley de cosenos:

$$cos\theta_3 = \frac{J^2 - L_2^2 - L_3^2}{2 * L_2 * L_3}$$

$$\theta_3 = cos^{-1}(cos\theta_3)$$

- Ángulo theta2 (rotación del hombro) usando la ley de senos:

$$\beta = tan^{-1}\left(\frac{L_3 * sen\theta_3}{L_2 + L_3 * cos\theta_3}\right)$$

$$\alpha = tan^{-1}\left(\frac{Z}{D}\right)$$

$$\theta_2 = \alpha - \beta$$

- Ángulo theta4 (rotación de la muñeca):

$$\theta_4 = 0 - (\theta_2 + \theta_3)$$

En el Anexo I están adjuntadas capturas de pantalla de la programación de las funciones expuestas realizada en Matlab.

En el caso del robot que se pretende crear en este trabajo, se ha diseñado un código en Matlab para implementar la cinemática. El código de Matlab consta de dos funciones principales, una en la que se representa la cinemática directa con los parámetros de Denavit-Hartenberg y otra en la que se desarrolla la cinemática inversa con los ángulos calculados mostrados anteriormente.

También se han diseñado otras dos funciones, para mostrar gráficamente el resultado del robot al aplicar la cinemática. Estas son para dibujar las líneas y el sistema de coordenadas. Esto se hizo para poder simular la cinemática y comprobar los valores que se obtenían cuando aún no se tenía el robot al 100% operativo y así poder asegurar que esta cinemática era correcta.

El programa funciona de la siguiente manera. Se le introduce un punto del plano al cual se quiere que llegue el efector final, es decir, la pinza, y posteriormente se ejecuta la función de la cinemática inversa. Esto devuelve un vector con 4 ángulos, uno para cada motor que son los que se debe mover el robot para llegar con la pinza al punto elegido. Al mismo tiempo se produce una representación gráfica del robot en Matlab, como se puede apreciar en las siguientes imágenes:

```
Command Window
>> P=[0.33 0 0]

P =

    0.3300         0         0

>> cin_inv(P)

ans =

    1.0000         0         0    0.3168
         0    0.0000   -1.0000   -0.0000
         0    1.0000    0.0000    0.0073
         0         0         0    1.0000


ans =

         0  -28.9264         0   28.9264
```

Figura 32: Indicación de un punto en la ventana de comandos de Matlab.

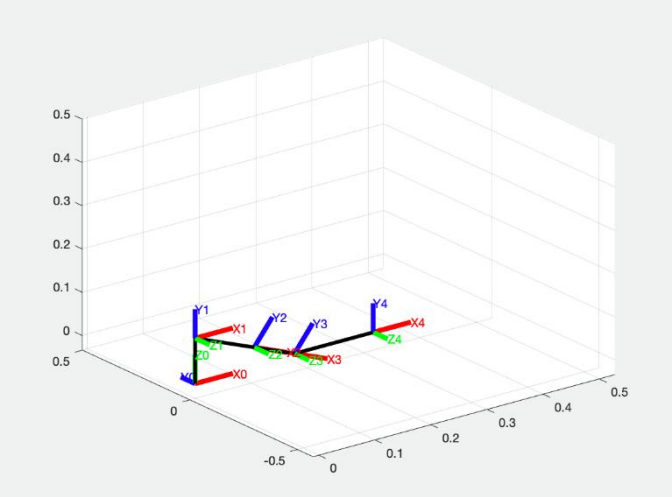


Figura 33: Representación gráfico del punto comandado en Matlab.

4 PROGRAMACIÓN DEL ARDUINO

En este apartado se comenta la programación llevada a cabo en el Arduino. Con el objetivo de no hacer muy extenso el apartado, se ha optado por comentar los problemas encontrados durante la programación de la realimentación y las soluciones propuestas. En el Anexo II se encuentra la programación realizada.

4.1 PROGRAMACIÓN DE LA REALIMENTACIÓN

La programación de la realimentación se ha basado en el siguiente diagrama de control:

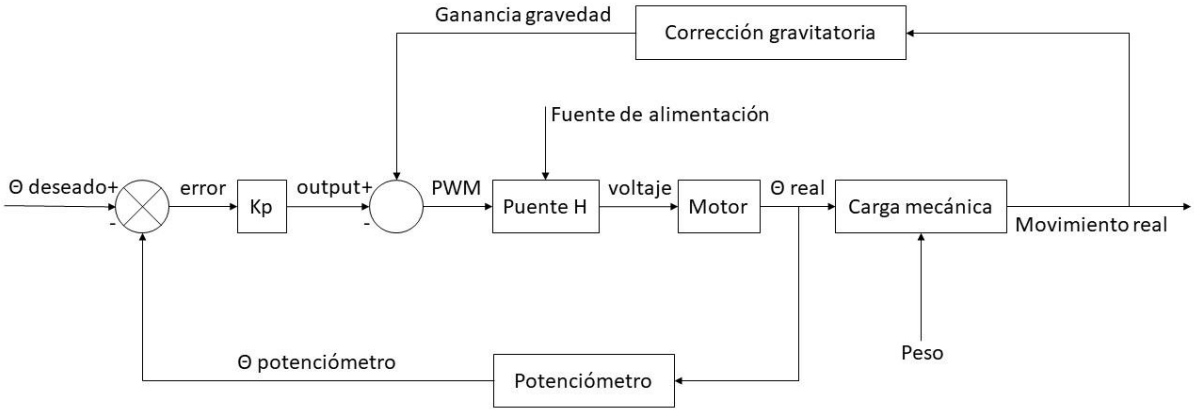


Figura 34: Diagrama de control de la realimentación.

Como se puede apreciar, se trata de un diagrama de control usual pero con la diferencia de que se le ha añadido una corrección gravitatoria.

Esto es debido a que cuando se realizan las pruebas de realimentación del hombro, se observa que el output, la respuesta del sistema realimentado que sirve para corregir el problema o controlar el sistema real, la solución es ineficiente o inadecuada. Esto se observa en la oscilación que produce, impidiendo encontrar un punto de equilibrio óptimo. Tras revisar el sistema de realimentación, se propone otra hipótesis y se descubre que hay más factores que afectan al sistema real, como por ejemplo, la gravedad.

Entonces se plantea el diagrama de la figura 31, con dos lazos de control, uno cerrado y otro abierto para la gravedad. Se crea otra ganancia, una ganancia gravitacional que depende del ángulo en el que está el motor del hombro, corrigiendo el sistema para que dependa del ángulo y, con ello, se aumenta proporcionalmente la corriente, para que el motor sea capaz de soportar el peso de la estructura.

4.1.1 Problemas encontrados durante la programación de la realimentación de los servomotores

Durante la programación de la realimentación de los servomotores se encuentran una serie de problemas que se describen en las gráficas de las siguientes imágenes.

En primer lugar, se especifica el significado de cada una de las líneas:

- Línea roja: corresponde al output, la señal a corregir.
- Línea verde: corresponde al resultado real.
- Línea azul: corresponde a los ángulos que se están mandando al resultado esperado.

En esta primera gráfica el output es de más de 1600 puntos lo que ocasiona errores en la realimentación. Para solucionarlo se ajustan los valores de las constantes de control.

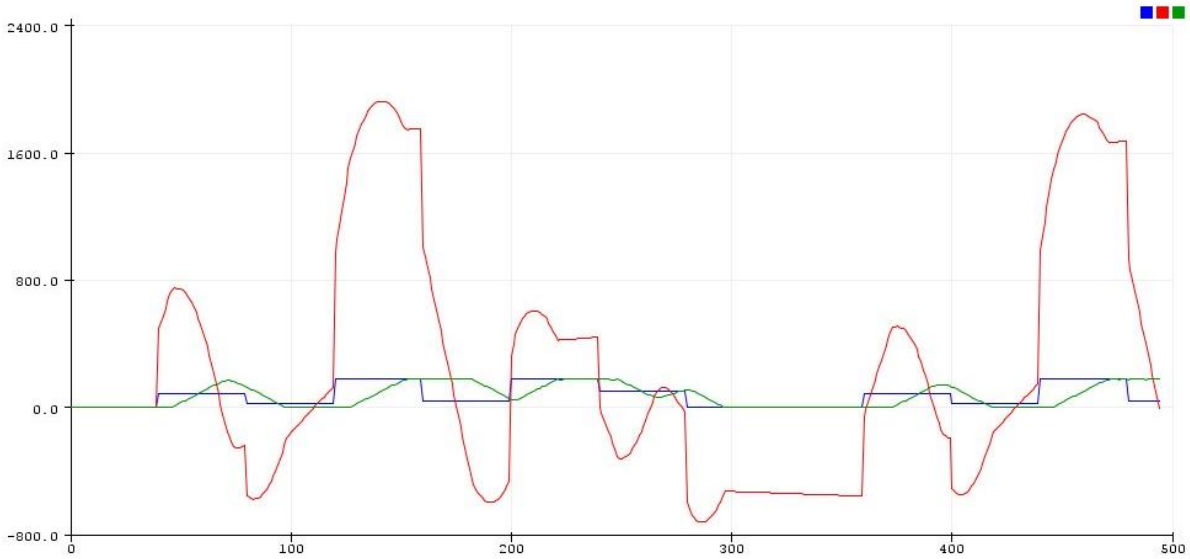


Figura 35: Gráfica de fallo en la realimentación del servomotor.

En la segunda gráfica la línea roja tiende a + ∞, lo que supone un error de convergencia, es decir, indica un error del sistema dinámico en alguna constante del controlador PID. El origen de este problema está en que el error se va acumulando gradualmente por alguno de los parámetros PID y por tanto nunca acabará convergiendo. Para solucionar esto hemos cambiado los valores de dichos parámetros

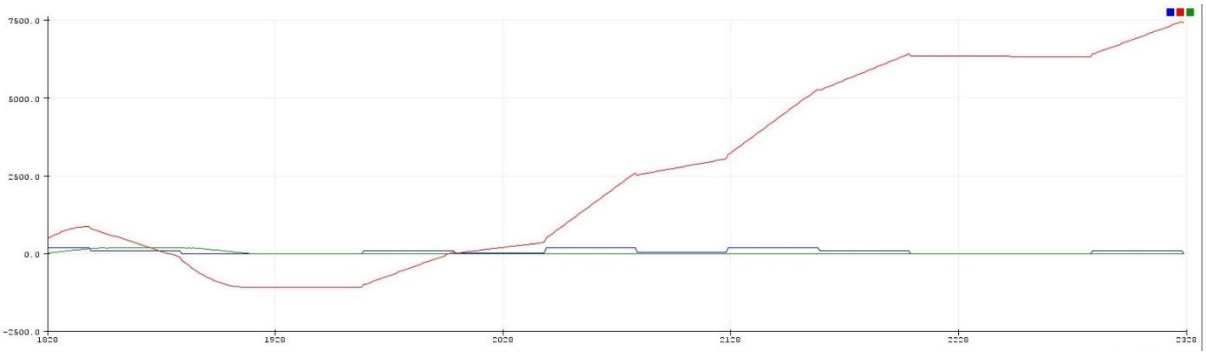


Figura 36: Gráfica de error de convergencia en la realimentación del servomotor.

A medida que se van aplicando cambios y mejoras, las gráficas muestran resultados más favorables. Una de las mejores respuestas obtenidas es la que se muestra en la siguiente imagen, donde la línea verde representa el ángulo real del motor y la azul el ángulo que se quiere alcanzar.

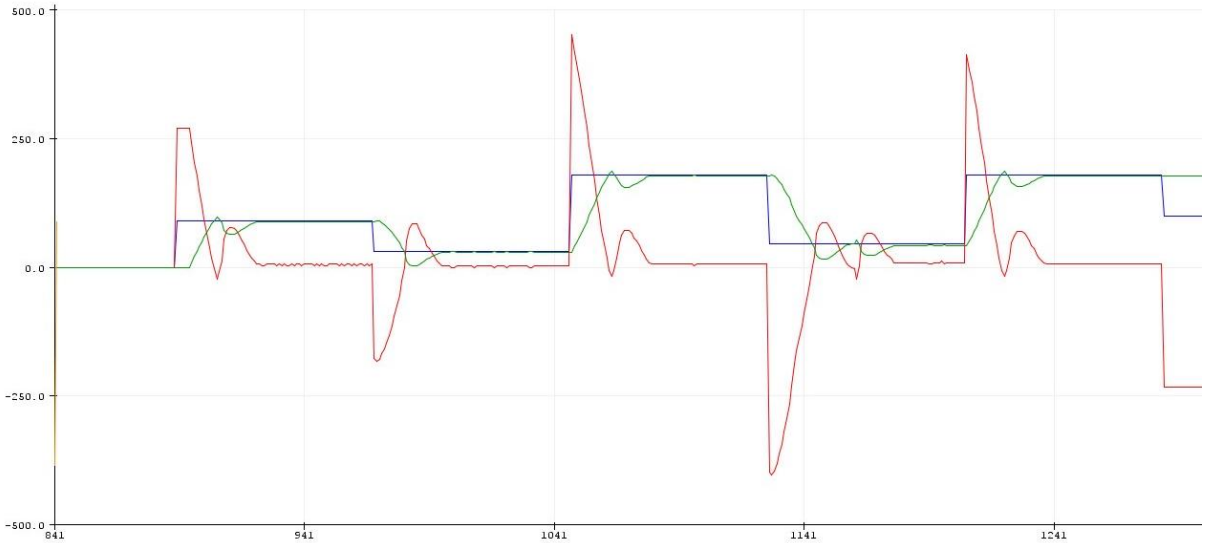


Figura 37: Gráfica con mejora en la realimentación del servomotor.

Añadiendo un lazo de control más que corrige la gravedad (línea amarilla) se obtiene un resultado mejor, siendo la línea verde la respuesta del sistema y la roja el ángulo real del motor.

Figura 38: Gráfica de la realimentación del servomotor con un lazo de control adicional.

Con esto se quiere demostrar que la parte más difícil de la realimentación de los servomotores es hacer coincidir los valores de lectura digitales con la realidad.

4.1.2 Problemas encontrados durante la programación de la realimentación del codo

La realimentación del codo también ocasiona problemas, volviéndose inviable dado que se convierte en un sistema de control de tercer grado y, por lo tanto, no resulta viable programarlo.

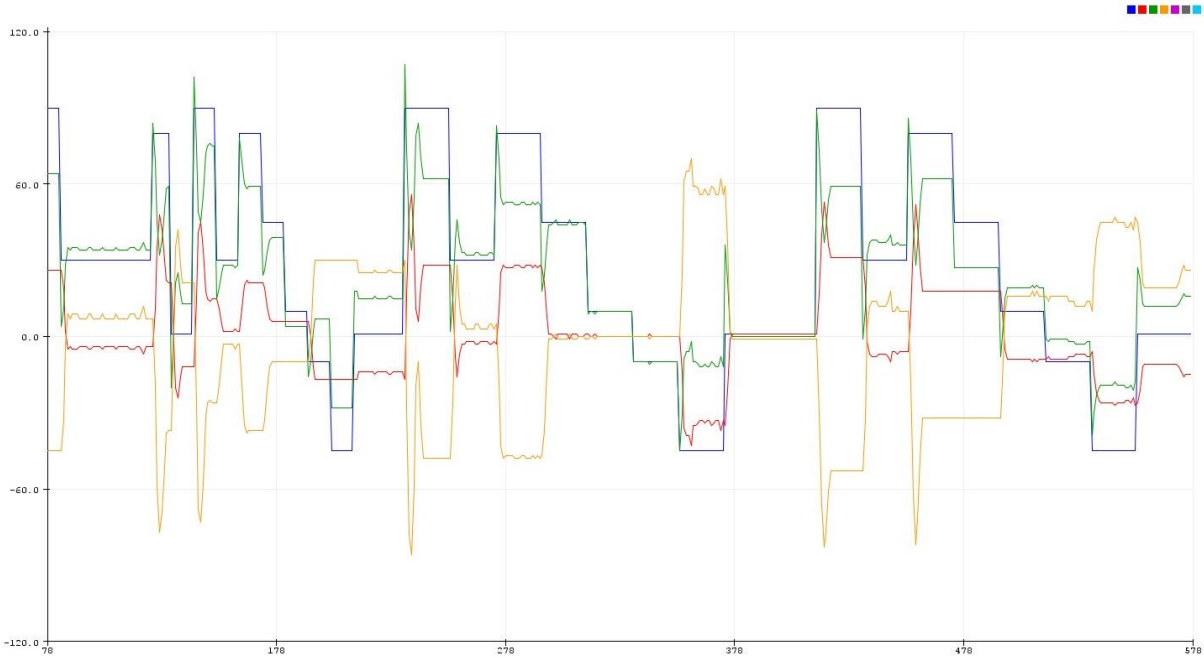


Figura 39: Gráfica de fallos en la realimentación del codo.

En su lugar se opta por realimentar el codo que gira en la base, obteniendo mejores resultados como se puede apreciar en la siguiente imagen.

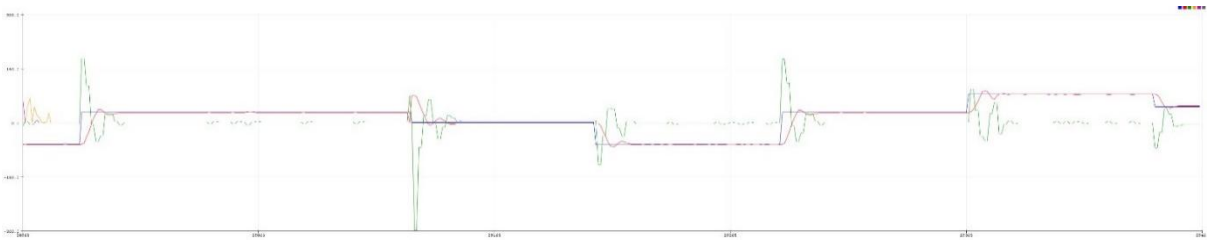


Figura 40: Gráfica de fallos en la realimentación del codo de la base giratoria.

4.1.3 Problemas encontrados durante la programación de la realimentación de la base

A diferencia del lazo de control del hombro, que al estar constantemente levantando peso, además de llevar una parte gravitatoria para contrarrestar la gravedad, para que se mantenga en la misma posición, teniendo que estar constantemente aportando potencia, de manera que no se caiga el brazo, el lazo de control de la base, únicamente se tiene que ejercer fuerza cuando realiza un cambio y, por lo tanto, no tiene que estar ejerciendo potencia constantemente del motor.

Además, como se puede apreciar en la gráfica de la siguiente imagen, entre las líneas roja (ángulo real) y azul (ángulo deseado) hay un error de unos 10 a 15 grados de diferencia. Esto se debe a que, como se comenta anteriormente, el output genera más potencia de la que realmente necesita el motor. Esta no es acumulativa, se trata de una aplicación puntual de fuerza para moverlo y no un añadido de la fuerza que se estaba ejerciendo antes.

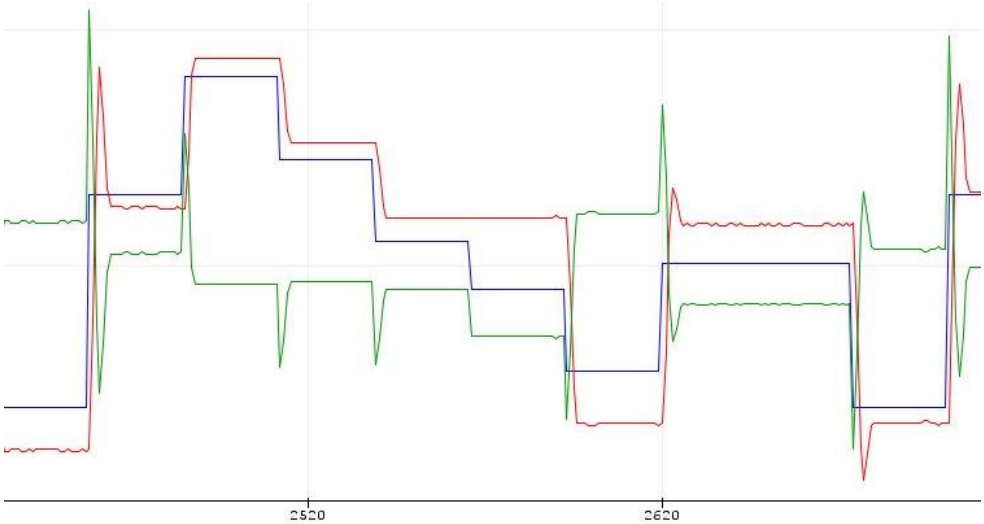


Figura 41: Gráfica de fallos en la realimentación de la base.

El delay genera problemas en el control. Este mantiene un impulso durante un tiempo de milisegundos, lo que genera que el impulso sea mucho más grande que el necesario, originando oscilaciones demasiado grandes.

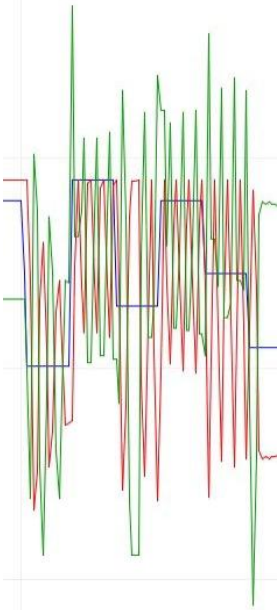


Figura 42: Gráfica del delay de la base.

5 COSTES

En este apartado se presentan los costes de cada elemento utilizado en la construcción del robot y el coste total del proyecto.

En primer lugar, los costes asociados a la impresión de las piezas del robot.

Tabla 10: Costes de la impresión de las piezas del robot.

Elemento	Nombre	Coste unitario	Unidades	Coste total
1	Base rotatoria	4,21 €	1	4,21 €
2	Base	15,83 €	1	15,83 €
3	Base pinza	0,50 €	1	0,50 €
4	Brazo 1 derecho	2,48 €	1	2,48 €
5	Brazo 1 izquierdo	2,05 €	1	2,05 €
6	Brazo 2 derecho	1,25 €	1	1,25 €
7	Brazo 2 izquierdo	1,00 €	1	1,00 €
8	Eje del brazo 2	0,50 €	1	0,50 €
9	Anclaje derecho	3,86 €	1	3,86 €
10	Anclaje izquierdo	2,40 €	1	2,40 €
11	Engranaje grande 1	0,50 €	1	0,50 €
12	Engranaje grande 2	1,17 €	1	1,17 €
13	Engranaje pequeño	0,50 €	1	0,50 €
14	Muñeca	2,22 €	1	2,22 €
15	Pinza barra	0,50 €	2	1,00 €
16	Pinza engranaje	0,50 €	2	1,00 €
17	Pinza extremo	0,50 €	2	1,00 €
				41,47 €

En segundo lugar, los costes de los elementos electrónicos del robot.

Tabla 11: Costes de los elementos electrónicos.

Elemento	Nombre	Coste unitario	Unidades	Coste total
1	Servomotor DM996	6,24 €	3	18,72 €
2	Conector de acoplamiento de brida	2,74 €	1	2,74 €
3	Placa controladora de accionamiento	3,49 €	1	3,49 €
4	Servomotor SG90	2,99 €	2	5,98 €
5	Motor paso a paso 28BYJ-48	2,79 €	1	2,79 €
				33,72 €

Por último, los costes asociados a otros materiales necesarios para la construcción del robot y las piezas que tiene que coger.

Tabla 12: Costes de materiales adicionales para el montaje del robot.

Elemento	Nombre	Coste unitario	Unidades	Coste total
1	Ejes	6,80 €	1	6,80 €
2	Caja de tornillos M3x10	3,10 €	1	3,10 €
				9,90 €

Teniendo en cuenta los costes de las piezas del robot impresas, los elementos electrónicos y los materiales adicionales, el coste total asciende a 85,09 €.

6 REFERENCIAS

[1] C. Urdiales, A. Bandera y F. Sandoval, «*Historia y tendencias actuales de la robótica*», Departamento de Tecnología Electrónica E.T.S.I. Telecomunicación, Universidad de Málaga. http://www.grupoisis.uma.es/isis/administrator/components/com_jresearch/files/publications/electronico.PDF

[2] C. Rodríguez Perez, «*Diseño De Un Robot Antropomórfico De Propósito General*», Documento de proyecto, Universidad Tecnológica de Pereira, Colombia, 2013. https://repositorio.utp.edu.co/server/api/core/bitstreams/8d249eca-bc35-43e7-b4a5-54cda919cb3d/content

[3] F. Araujo, L. Ayala y C. Bermeo, «*Robots Antropomórficos*», Universidad Politécnica Salesiana. https://arhatarahant.wordpress.com/wp-content/uploads/2014/07/robots_antropomorficos.pdf

[4] J. Legarreta y R. Martinez, «*Tema 5. Modelado Geométrico Y Cinemático Del Robot*», Universidad de País Vasco-Euskal Herriko Unibertsitatea, Bilbao, Presentación. https://ocw.ehu.eus/pluginfile.php/50445/mod_resource/content/8/T5%20CINEMATICA%20OCW_Revision.pdf

ANEXO I

```
%Funcion calcular matrices dh ingresando theta, d, a, alfa

function [ A40 ] = parDH( theta, d,a,alfa)

    A40 = [cos(theta),(-cos(alfa)*sin(theta)),(sin(alfa)*sin(theta)),(a*cos(theta));...
        sin(theta),(cos(alfa)*cos(theta)),(-sin(alfa)*cos(theta)),(a*sin(theta));...
        0,sin(alfa),cos(alfa),d;0,0,0,1];

end

% Dibujar una linea a partir de dos puntos

function linea( p0, p1, color, grosor )
% vectores que contienen las componentes de los puntos a dibujar
x = [ p0(1), p1(1)];
y = [ p0(2), p1(2)];
z = [ p0(3), p1(3)];

% dibujo de las lineas
plot3(x,y,z, 'Color', color ,'LineWidth',grosor);

grid on


%Función que sirve para dibujar un sistema de coordenadas
%teniendo una matriz de transformación homogenea (A) y una longitud t

function sist_coor(A,t,E)
    a1=[0;0;0;1];
    b1=[t;0;0;1];
    c1 = [0;t;0;1];
    d1 = [0;0;t;1];

    a0 = A*a1;
    b0 = A*b1;
    c0 = A*c1;
    d0 = A*d1;

    rojo =[1, 0, 0];
    verde = [0, 1, 0];
    azul = [0, 0,1];

    linea(a0,b0,rojo,4);
    hold on
    linea(a0,c0,azul,4);
    hold on
    linea(a0,d0,verde,4);
    hold on

     axis([-15 45 -15 45 -15 45])

 text(b0(1), b0(2), b0(3), E(1,1:2), 'Color', rojo);
    text(c0(1), c0(2), c0(3), E(1,3:4), 'Color', azul);
    text(d0(1), d0(2), d0(3), E(1,5:6), 'Color', verde);
```

```
%Funcion parámetros cinemática directa

function [A] = cin_direc(q)
% recibe el vector de los angulos de las articulaciones
L1=0.105;
L2=0.12;
L3=0.082;
L4=0.14;
Lt = L1 + L2 + L3 +L4;

A10=parDH((q(1)*pi)/180,L1,0,(pi/2));
A21=parDH((q(2)*pi)/180,0,L2,0);
A32=parDH(((q(3)*pi)/180),0,L3,0);
A43=parDH((q(4)*pi)/180,0,L4,0);

A20 = A10*A21;
A30 = A10*A21*A32;
A40 = A10*A21*A32*A43;

A = A40;
A0 = eye(4);

sist_coor(A0,0.15*Lt,['X0' 'Y0' 'Z0']);
hold on
sist_coor(A10,0.15*Lt,['X1' 'Y1' 'Z1']);
hold on
sist_coor(A20,0.15*Lt,['X2','Y2','Z2']);
hold on
sist_coor(A30,0.15*Lt,['X3','Y3','Z3']);
hold on
sist_coor(A40,0.15*Lt,['X4','Y4','Z4']);
hold on


linea([0 0 0], A10(1:3,4), [0 0 0], 3);
hold on
linea(A10(1:3,4), A20(1:3,4), [0 0 0], 3);
hold on
linea(A20(1:3,4), A30(1:3,4), [0 0 0], 3);
hold on
linea(A30(1:3,4), A40(1:3,4), [0 0 0], 3);
hold on

axis([0 Lt + (0.2*Lt), -(Lt + (0.2*Lt)) Lt + (0.2*Lt), 0, Lt + (0.2*Lt)])
end
```

```
%Esta funcion recibe como parametros angulos, para luego calcular la posicion del efector final,
% y dibujar esa posicion.

function [q] = cin_inv(P)


%calcula cinematica inversa con un punto x,y,z
L1=0.105;
L2=0.12;
L3=0.082;
L4=0.14;

    % ángulo theta1 (rotación en la base):
    q1 = atan2(P(2), P(1));

    % distancia horizontal desde la base al punto proyectado en XY:
    D = sqrt(P(1)^2 + P(2)^2) - L4 * cosd(0);

    % altura vertical desde la base al punto P menos la proyección de L4:
    Z = P(3) - L1 - L4 * sind(0);

    % distancia total desde la base al punto final P:
    J = sqrt(D^2 + Z^2);

    % ángulo theta3 (rotación del codo):
    cos_q3 = (J^2 - L2^2 - L3^2) / (2 * L2 * L3);
    cos_q3 = min(max(cos_q3, -1), 1);
    q3 = acos(cos_q3);

    % ángulo theta2 (rotación del hombro):
    beta = atan2(L3 * sin(q3), L2 + L3 * cos(q3));
    alfa = atan2(Z, D);
    q2 = alfa - beta;

    % cálculo del ángulo theta4 (rotación de la muñeca)
    q4 = 0 - (q2 + q3);

    % limitar los ángulos entre -180 y 180 grados:
    q1 = mod(q1 + 180, 360) - 180;
    q2 = mod(q2 + 180, 360) - 180;
    q3 = mod(q3 + 180, 360) - 180;
    q4 = mod(q4 + 180, 360) - 180;



% convertir los ángulos de radianes a grados
q1 = q1 * 180 / pi;
q2 = q2 * 180 / pi;
q3 = q3 * 180 / pi;
q4 = q4 * 180 / pi;

%vector con todos los ángulos
q = [q1 q2 q3 q4];


cin_direc(q)
end
```

ANEXO II

```
#include <Servo.h>
Servo MotorCodo;  // Crea un objeto de la clase Servo
Servo MotorMuneca;  // Crea un objeto de la clase Servo
Servo Pinza;

 class realMotor { //clase para almacenar los valores de cada MOTOR
REALIMENTADO
public:
                      // Parámetros del PID
    double Kp = 0.0;
    double Ki = 0.0;
    double Kd = 0.0;
              double P, I = 0, D;  // Inicializa I y D
              double output, error, lastInput = 0;
              unsigned long lastTime = 0;

                      // Parámetros del Hardware
    int pinPotenciometro;  // Pin para el potenciómetro
    int entradaA;          // Primer pin de control para el puente H
    int entradaB;          // Segundo pin de control para el puente H
    int enable;           // Pin de habilitación para el puente H
    int minvalpot = 0;     // Valor minimo de lectura del potenciometro
    int maxvalpot = 1023;  // Valor máximo de lectura del potenciometro
    int minangpot = 0;
    int maxangpot = 180;

                      //Parámetros del control
    int valorPotenciometro;
    int anguloReal;
    int anguloDeseado;
    int salidaFinal = 0;
    int absSalidaFinal = 0;

//                        ORDEN CONSTRUCTOR: entradaA, entradaB, enable,
pinPotenciometro, Kp, Ki, Kd, minvalpot, maxvalpot, minangpot, maxangpot

    // Constructor para inicializar todos los pines necesarios
    realMotor(int _entradaA, int _entradaB, int _enable, int
_pinPotenciometro, double _Kp, double _Ki, double _Kd, int _minvalpot, int
_maxvalpot, int _minangpot, int _maxangpot)
    : entradaA(_entradaA), entradaB(_entradaB), enable(_enable),
pinPotenciometro(_pinPotenciometro),
      Kp(_Kp), Ki(_Ki), Kd(_Kd), minvalpot(_minvalpot), maxvalpot(_maxvalpot),
minangpot(_minangpot), maxangpot(_maxangpot) {
        pinMode(pinPotenciometro, INPUT);  // Configura el pin del
potenciómetro como entrada
```

```
        pinMode(entradaA, OUTPUT);         // Configura el primer pin de
control como salida
        pinMode(entradaB, OUTPUT);         // Configura el segundo pin de
control como salida
        pinMode(enable, OUTPUT);           // Configura el pin de habilitación
como salida
    }

      void FuncionMotorBase(realMotor& motor){};
      void FuncionMotorHombro(realMotor& motor){};

    //   Función para manejar el control del motor mediante el puente H
    void manageMotor()
{                                                                // A tener
actualizado antes salidaFinal y absSalidaFinal

        // Control del motor mediante el puente H
        if (salidaFinal > 0) {
            digitalWrite(entradaA, HIGH);
            digitalWrite(entradaB, LOW);
        } else if (salidaFinal < 0) {
            digitalWrite(entradaA, LOW);
            digitalWrite(entradaB, HIGH);
        } else {
            digitalWrite(entradaA, LOW);
            digitalWrite(entradaB, LOW);
        }

        // Aplicar el ciclo de trabajo correcto
        analogWrite(enable,
absSalidaFinal);                                  //ejecutador
    }
};
const double gravComp_1 = 0.75;  // Compensación por gravedad (TORQUE GENERADO
POR LA GRAVEDAD EN FUNCION DEL ANGULO)
const double torqueMax = 1.47; // Torque máximo del motor en Nm

    class coordBloques {
public:
        // Variables para almacenar los ángulos finales del movimiento
        int CinInvAngBase;
        int CinInvAngHombro;
        int CinInvAngCodo;
        int CinInvAngMuneca;

        // Constructor para inicializar el objeto
        coordBloques(int base, int hombro, int codo, int muneca)
```

```
            : CinInvAngBase(base), CinInvAngHombro(hombro),
CinInvAngCodo(codo), CinInvAngMuneca(muneca) {}
    };


//                                                        CONSTRUCCION DE
OBJETOS:
// Creación del objeto motorHombro
// Parámetros: entradaA, entradaB, enable, pinPotenciometro, Kp, Ki, Kd,
minvalpot, maxvalpot, minangpot, maxangpot
realMotor motorHombro(
    5,       // entradaA: Primer pin de control para el puente H
    6,       // entradaB: Segundo pin de control para el puente H
    7,       // enable: Pin de habilitación para el puente H
    A0,      // pinPotenciometro: Pin para el potenciómetro
    4.0,     // Kp: Ganancia proporcional
    0.0,     // Ki: Ganancia integral
    0.0,     // Kd: Ganancia derivativa
    100,       // minvalpot: Valor mínimo de lectura del potenciómetro
    1000,    // maxvalpot: Valor máximo de lectura del potenciómetro
    -15,       // minangpot: Ángulo mínimo correspondiente al potenciómetro
    200      // maxangpot: Ángulo máximo correspondiente al potenciómetro
);

// Creación del objeto motorBase
// Parámetros: entradaA, entradaB, enable, pinPotenciometro, Kp, Ki, Kd,
minvalpot, maxvalpot, minangpot, maxangpot
realMotor motorBase(
    4,       // entradaA: Primer pin de control para el puente H
    3,       // entradaB: Segundo pin de control para el puente H
    2,       // enable: Pin de habilitación para el puente H
    A1,      // pinPotenciometro: Pin para el potenciómetro
    2.0,     // Kp: Ganancia proporcional
    0.0,     // Ki: Ganancia integral
    2.0,     // Kd: Ganancia derivativa
    0,       // minvalpot: Valor mínimo de lectura del potenciómetro
    1023,    // maxvalpot: Valor máximo de lectura del potenciómetro
    -105,    // minangpot: Ángulo mínimo correspondiente al potenciómetro
    90       // maxangpot: Ángulo máximo correspondiente al potenciómetro
);

//Coordenadas de los bloques que ya hemos trasladado desde
MATLAB                   //base, hombro, codo, muñeca
 /* coordBloques lejosP0(-70, 30, -45, -40);
  coordBloques lejosP1(-70, 30, -38, -30);
  coordBloques medioP0(-70, 45, -58, -70);
  coordBloques medioP1(-70, 80, -65, -60);
  coordBloques cercaP0(-70, 70, -90, -69);
  coordBloques cercaP1(-70, 90, -75, -80);*/
```

```
  coordBloques bloques[] = {
        {0, 30, 90, 90}, //ORIGEN
        {-70, 30, -45, -40},  // lejosP0
        {-70, 45, -58, -70},  // medioP0
        {-70, 70, -90, -69},  // cercaP0
        {-70, 30, -38, -30},  // lejosP1
        {-70, 80, -65, -60},  // medioP1
        {-70, 90, -75, -80},  // cercaP1
    };
  coordBloques ORIGEN (0, 30, 90, 90);
  coordBloques recogerBloque(85, 30, -50, -20);


//                                                        Constantes y
variables globales

const unsigned long updateInterval = 8000;  // Intervalo de actualización en
milisegundos
bool MovimientoBloque = false; //variable que activa la cinematica del
movimiento
bool cogiendoBloque = false, dejandoBloque = false;
bool comunicacionMatlab = false;

////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////
int CinInvAngBase;
int CinInvAngHombro;
int CinInvAngCodo;
int CinInvAngMuneca;
////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////

int indiceBloque = 0;
int etapa=0;
unsigned long tiempoInicio = 0;
unsigned long tiempoRonda = 0;

void setup() {
    Serial.begin(9600);
    MotorCodo.attach(12);  // Asigna el pin 9 a la señal de control del servo
    MotorMuneca.attach(10);  // Asigna el pin 9 a la señal de control del
servo
    Pinza.attach(11); // Vincula el servo al pin 11

     tiempoInicio = millis();
     coordBloques ORIGEN (0, 30, 90, 90);
     coordBloques recogerBloque(85, 30, -50, -20);
```

```
     // Posicionar en Origen
       /*   CinInvAngBase = ORIGEN.CinInvAngBase;
          CinInvAngHombro = ORIGEN.CinInvAngHombro;
          CinInvAngCodo = ORIGEN.CinInvAngCodo;
          CinInvAngMuneca = ORIGEN.CinInvAngMuneca;
*/
          CinInvAngBase = 0;
          CinInvAngHombro = 30;
          CinInvAngCodo = 90;
          CinInvAngMuneca = 90;

}

void loop() {
////////////////////////////////////////////////////////////////////////////
////////////////////

    //      FuncionMotorBase(motorBase);
      //    FuncionMotorHombro(motorHombro);
        //  updateAngObjetivo_Codo_Muneca(CinInvAngCodo,CinInvAngMuneca);
          //MotorCodo.write(CinInvAngCodo);   // Mueve el servo a la posición
inicial (0 grados)
          //MotorMuneca.write(CinInvAngMuneca);   // Mueve el servo a la
posición inicial (0 grados)

      tiempoRonda=millis();
          //COMUNICACION CON MATLAB
          comunicacionMatlab=true;

    if (comunicacionMatlab){ //Por aqui recibiriamos a traves de un bucle la
informacion desde MATLAB


      if ((tiempoRonda - tiempoInicio > 2000) && etapa==0) { //pongo un tiempo
de inicializacion de variables

          // Posicionar en Origen
          CinInvAngBase = ORIGEN.CinInvAngBase;
          CinInvAngHombro = ORIGEN.CinInvAngHombro;
          CinInvAngCodo = ORIGEN.CinInvAngCodo;
          CinInvAngMuneca = ORIGEN.CinInvAngMuneca;
          abrirPinza();

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }
```

```
        else if((tiempoRonda - tiempoInicio > 37000) && etapa==1) { //5
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)

          // Acceder a las variables del objeto actual
          CinInvAngBase = recogerBloque.CinInvAngBase;
          CinInvAngHombro = recogerBloque.CinInvAngHombro;
          CinInvAngCodo = recogerBloque.CinInvAngCodo;
          CinInvAngMuneca = recogerBloque.CinInvAngMuneca;
          CinInvAngHombro += 30;

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }

        else if ((tiempoRonda - tiempoInicio > 10000) && etapa==2) { //3
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)
          CinInvAngHombro -= 30;

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }

     else if ((tiempoRonda - tiempoInicio > 13000) && etapa==3) { //3 segundos
y cierra la pinza, luego otros 2 segundos de esper
          cerrarPinza();
          etapa++;
        }

       else if ((tiempoRonda - tiempoInicio > 15000) && etapa==4) { //va a
dejar el bloque

          CinInvAngBase = bloques[indiceBloque].CinInvAngBase;
          CinInvAngHombro = bloques[indiceBloque].CinInvAngHombro;
          CinInvAngCodo = bloques[indiceBloque].CinInvAngCodo;
          CinInvAngMuneca = bloques[indiceBloque].CinInvAngMuneca;
          CinInvAngHombro += 30;

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }
```

```
      else if ((tiempoRonda - tiempoInicio > 19000) && etapa==5) { //4
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)
          CinInvAngHombro -= 30;

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }

      else if ((tiempoRonda - tiempoInicio > 22000) && etapa==6) { //3
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)
          abrirPinza();
          etapa++;
        }

     else if ((tiempoRonda - tiempoInicio > 26000) && etapa==7) { //4
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)
          CinInvAngHombro += 30;

          motorBase.anguloDeseado = CinInvAngBase;
          motorHombro.anguloDeseado = CinInvAngHombro;
          etapa++;
        }

         else if ((tiempoRonda - tiempoInicio > 30000) && etapa==8){ //4
segundos y se mueve a por un bloque (realmente se situa 30 grados por encima
para no chocarse con nada)
          // Incrementar el índice para pasar al siguiente objeto en el
próximo ciclo
                 indiceBloque++;
                tiempoInicio=millis();
                etapa++;
        };


          // Si el índice es mayor o igual al tamaño del vector, reiniciarlo
          if (indiceBloque >= sizeof(bloques) / sizeof(bloques[0])) {
              indiceBloque = 0;
              etapa=0;
          }




         FuncionMotorBase(motorBase);
         FuncionMotorHombro(motorHombro);
```

```
        updateAngObjetivo_Codo_Muneca(CinInvAngCodo,CinInvAngMuneca);
        MotorCodo.write(CinInvAngCodo);   // Mueve el servo a la posición
inicial (0 grados)
        MotorMuneca.write(CinInvAngMuneca);   // Mueve el servo a la posición
inicial (0 grados)
    }



                //ACTUALIZO MONITOREO
            actualizarRealimentacion();



// Pequeña pausa para estabilizar, ajustable
//delay(50);
}

void actualizarRealimentacion(){
          Serial.print("\t");
          Serial.print(motorBase.anguloDeseado);
          Serial.print("\t");
          Serial.print(motorBase.anguloReal);
          Serial.print("\t");
          Serial.print(motorBase.output);
          Serial.print("\t");

           Serial.print("\t");
          Serial.print(motorHombro.anguloDeseado);
          Serial.print("\t");
          Serial.print(motorHombro.anguloReal);
          Serial.print("\t");
          Serial.println(motorHombro.output);
}

void FuncionMotorHombro(realMotor& motor){ //Funcion para controlar al
servo                                   DEL HOMBRO

    motor.valorPotenciometro = analogRead(motor.pinPotenciometro);
    // Lee el valor actual del potenciómetro y conviértelo a grados
    motor.anguloReal = map(motor.valorPotenciometro, motor.minvalpot,
motor.maxvalpot, motor.minangpot, motor.maxangpot); // asignacion de angulos
corrigiendo los problemas del sensor

    // Calcula el error
    motor.error = motor.anguloDeseado - motor.anguloReal;
    // Calcula la señal de control del primer lazo (cerrado)
    motor.output = motor.Kp * motor.error;
```

```
    //Añadimos el componente del lazo de realimentación gravitatorio a la
salida del sistema
    double gan_gravit = calcularCompensacionGravedad(motor.anguloReal);
    double salidaReal= motor.output + gan_gravit;

    // Toma el valor absoluto del output para PWM
    int absSalidaReal = abs(int(salidaReal));
    // Limita el valor a un máximo de 255 para la señal PWM
    absSalidaReal = constrain(absSalidaReal, 0, 255);

    // Actualiza el valor de salida cada x miliseconds      ->  EN EL
OBJETO  <-
    static unsigned long tiempoactualizacion = 0;
    if (millis() - tiempoactualizacion >= 5) {
         motor.salidaFinal=salidaReal;
         motor.absSalidaFinal=absSalidaReal;
        tiempoactualizacion = millis();
    }



      //APLICO EL OUTPUT AL PUENTE H
      motor.manageMotor();
      actualizarRealimentacion();
}




void FuncionMotorBase(realMotor& motor){ //Funcion para controlar al
servo                                          DE LA BASE

    motor.valorPotenciometro = analogRead(motor.pinPotenciometro);
    // Lee el valor actual del potenciómetro y conviértelo a grados
    motor.anguloReal = map(motor.valorPotenciometro, motor.minvalpot,
motor.maxvalpot, motor.minangpot, motor.maxangpot); // asignacion de angulos
corrigiendo los problemas del sensor

    // Calcula el error
    motor.error = motor.anguloDeseado - motor.anguloReal;
    // Calcula la señal de control del primer lazo (cerrado)
    motor.P = motor.Kp * motor.error;

    // Computar PID como normal
        unsigned long now = millis();
        double timeChange = (double)(now - motor.lastTime);

        if (timeChange >= 10) {  // Intervalo de cálculo, ajusta según
necesites
```

```
            // Calcular el término integral
            motor.I += (motor.Ki * motor.error);
            motor.I = constrain(motor.I, 0, 255);  // Evita el wind-up del
integrador

            // Calcular el término derivativo
            motor.D = (motor.anguloReal - motor.lastInput);

            // Calcular salida PID
            motor.output = motor.P  - motor.Kd * motor.D; //+motor.I;


            // Recordar algunos valores para la próxima vez
            motor.lastInput = motor.anguloReal;
            motor.lastTime = now;
        }

    motor.salidaFinal = motor.output;

    // Actualiza el valor de salida                              ->  EN EL
OBJETO  <-
    // Toma el valor absoluto del output para PWM
    motor.absSalidaFinal = abs(int(motor.salidaFinal));

    //Con esta linea estoy mandando un pulso proporcional al error real
      motor.absSalidaFinal = map(motor.absSalidaFinal, 0, 100, 0, 200);

    // Limita el valor a un máximo de 255 para la señal PWM
    motor.absSalidaFinal = constrain(motor.absSalidaFinal, 0, 255);

      //APLICO EL OUTPUT AL PUENTE H
      motor.manageMotor();
      actualizarRealimentacion();
}





                                        //  Función para calcular la
compensación de torque por gravedad
double calcularCompensacionGravedad(double angulo) {
    // Convertir grados a radianes
    double radianes = angulo * (PI / 180.0);
    double componentegravedad = gravComp_1 * cos(radianes); //calculo el
torque con el componente del peso dado el angulo
    int correccingravedad = (int)(componentegravedad * (255 / 1.47));
```

```
    // Limitar el valor a rango PWM (-255 a 255)
    correcciongravedad = constrain(correcciongravedad, -255, 255);

    return correcciongravedad;
}

void abrirPinza() {
  Pinza.write(80); // Mueve el servo a 80 grados (abrir)
}

void cerrarPinza() {
  Pinza.write(0); // Mueve el servo a 0 grados (cerrar)
}


////////////////////////////////////////////////////////////////////////
ACTUALIZAR ANGULOS
//////////////////////////////////////////////////////////

      // Función para actualizar el ángulo objetivo de motorBase
void updateAngObjetivoBase(){
    motorBase.anguloDeseado = CinInvAngBase;
}

// Función para actualizar el ángulo objetivo de motorHombro
void updateAngObjetivoHombro() {
  motorHombro.anguloDeseado = CinInvAngHombro;
}

//Funcion para adaptar los angulos de la cinematica al SR fisico del robot
void updateAngObjetivo_Codo_Muneca(int AnguloDeseadoCodo,int
AnguloDeseadoMuneca){

    CinInvAngCodo = map(AnguloDeseadoCodo,90,-90,0,180);
    CinInvAngMuneca=AnguloDeseadoMuneca + 90;
}

////////////////////////////////////////////////////////////////////////
////////////////////
```